

UNIVERSITÀ TELEMATICA INTERNAZIONALE UNINETTUNO

FACOLTÀ DI INGEGNERIA

Corso di Laurea Magistrale in
Ingegneria Informatica

ELABORATO FINALE

in

Progettazione del software

Sviluppo di una Pipeline di Big Data Analytics per la Classificazione in Tempo Reale di Aritmie Cardiache

CORRELATORE
Prof. Mazzei Mauro

CANDIDATO
Donadello Andrea

TODO Dedicar

Sommario

Il monitoraggio continuo dell'elettrocardiogramma impone di classificare ogni battito mentre il segnale arriva, rispettando vincoli di latenza e affidabilità. Molti lavori sulla classificazione automatica si limitano all'addestramento e alla valutazione offline, senza collegare ingestione, inferenza e persistenza in un'architettura eseguibile.

Si propone una pipeline end-to-end per la classificazione *near-real-time* di battiti ECG, con approccio ibrido batch/streaming. Un `RandomForestClassifier`, addestrato in batch su feature a livello di battito dal dataset MIT-BIH in formato tabulare, viene distribuito come artefatto e applicato in streaming da Apache Spark Structured Streaming su eventi Kafka pubblicati da un simulatore edge. Le predizioni sono scritte su Delta Lake con checkpointing, garantendo recupero dopo failure e disaccoppiamento tra producer e consumer. L'implementazione in Scala è riproducibile con Docker e `sbt`.

Sul test set si osservano accuratezza globale circa al 96% e latenza end-to-end tipicamente sub-secondo, con limiti sulle classi rare e implicazioni cliniche degli errori di classificazione analizzate nel contesto delle super-classi AAMI. Il significato del lavoro è dimostrare che Kafka, Spark e Delta Lake possono comporsi in un sistema modulare per dati biomedici strutturati, fungendo da template per pipeline e-Health che uniscano qualità predittiva e proprietà non funzionali di un deployment distribuito.

Indice

Sommario	ii
1 Introduzione	1
1.1 Contesto e motivazione	1
1.2 Approccio e scelte tecnologiche	2
1.3 Obiettivi	3
1.4 Contributi	3
1.5 Struttura della tesi	4
2 Dominio Applicativo e Dataset	5
2.1 Dominio applicativo: il segnale ECG	5
2.2 Dataset sperimentale: MIT-BIH tabellare (Kaggle)	9
2.3 Edge Computing e ruolo del simulatore	12
2.4 Considerazioni etiche e implicazioni cliniche	14
3 Stack Tecnologico	16
3.1 Il linguaggio Scala: programmazione funzionale applicata ai Big Data	16
3.2 Apache Spark: Core, Spark SQL e MLlib	17
3.3 Apache Kafka: architettura publish-subscribe e disaccoppiamento	20
3.4 Delta Lake: il paradigma Data Lakehouse	22
3.5 Sintesi: requisiti e tecnologie adottate	23

4	Caso di studio	24
4.1	Pattern architetturali: Lambda, Kappa e posizionamento del progetto	24
4.2	Panoramica della pipeline	26
4.3	Vista fisica e deployment	27
4.4	Fase di addestramento: <code>EcgModelTrainer</code>	28
4.5	Simulazione IoT: <code>EcgPatientSimulator</code>	29
4.6	Inferenza in tempo reale: <code>EcgStreamingPredictor</code>	30
4.7	Orchestrazione: <code>AppRunner</code>	32
4.8	Requisiti non funzionali	32
5	Implementazione e Dettagli Tecnici	35
5.1	Organizzazione del codice sorgente	35
5.2	Build e gestione delle dipendenze	36
5.3	Infrastruttura Docker e broker Kafka	37
5.4	Configurazione <code>SparkSession</code>	37
5.5	Data pipeline e machine learning	38
5.6	Gestione dello streaming	40
5.7	Storage e sink: Delta Lake	42
5.8	Producer Kafka del simulatore	43
5.9	Orchestrazione e ciclo di vita	43
5.10	Gestione errori e resilienza	44
5.11	Strategie di testing	44
5.12	Riepilogo implementativo	45
6	Analisi dei Risultati e Test	47
6.1	Protocollo sperimentale	47
6.2	Performance del modello	48

6.3	Analisi degli errori e impatto clinico	50
6.4	Analisi della latenza	51
6.5	Throughput e back-pressure	53
6.6	Scalabilità	54
6.7	Riproducibilità	54
6.8	Sintesi dei risultati sperimentali	55
7	Conclusioni	56
7.1	Sintesi del lavoro svolto e suo significato	56
7.2	Analisi comparativa e commento critico dei risultati	56
7.3	Parti omesse o non approfondite	57
7.4	Possibili ulteriori sviluppi	58

Elenco delle figure

2.1	Morfologia schematica di un battito ECG in ritmo sinusale: deflessioni P, Q, R, S e T; intervallo PR, durata del complesso QRS, segmento ST e intervallo QT. . .	6
3.1	Dal codice Scala al piano eseguito: Catalyst e Tungsten nel percorso di una query Spark SQL.	18
4.1	Schema concettuale dell'architettura Lambda: batch e speed layer alimentano il serving.	25
4.2	Flusso logico: training batch, ingestion simulata e inferenza streaming. Linee tratteggiate: orchestrazione AppRunner	26
4.3	Sequenza di avvio orchestrata da AppRunner : training condizionale, predictor in thread demone, attesa, simulatore, join.	27
4.4	Deployment mono-nodo: tre processi JVM Spark (<code>local[*]</code>) e un container Docker per Kafka.	27
6.1	F1-score per super-classe. Valori da sostituire con misure sperimentali definitive.	49

Elenco delle tabelle

2.1	Corrispondenza tra etichette nel CSV Kaggle e super-classi AAMI.	10
2.2	Raggruppamento AAMI delle annotazioni MIT-BIH (sintesi del contenuto clinico).	10
2.3	Feature per derivazione nel CSV MIT-BIH (prefissi 0_ e 1_).	11
3.1	Confronto tra Kafka con ZooKeeper e Kafka in modalità KRaft.	21
3.2	Mappatura requisiti-tecnologie.	23
4.1	Confronto sintetico Lambda, Kappa e impostazione del progetto.	25
5.1	Riepilogo componenti e scelte implementative.	45
6.1	Configurazione sperimentale (placeholder).	47
6.2	Metriche per super-classe AAMI (valori provvisori).	49
6.3	Decomposizione latenza end-to-end (ms, valori provvisori).	52

Capitolo 1

Introduzione

1.1 Contesto e motivazione

Le patologie cardiovascolari rappresentano una delle prime cause di morbidità e mortalità a livello globale; il monitoraggio continuo dell'attività cardiaca tramite segnali ECG è pertanto centrale per la diagnosi precoce e la gestione clinica.

In molti scenari applicativi, l'interpretazione del segnale non può essere affidata esclusivamente a una valutazione umana *ex post*, poiché il valore informativo del battito risiede anche nella *tempestività* con cui viene rilevata un'anomalia. Da questo punto di vista, l'analisi automatica dei battiti cardiaci pone sfide tecniche sostanziali:

- La necessità di elaborazione **near-real-time**,
- La gestione di flussi ad alta velocità,
- La robustezza rispetto a fault e perdita di pacchetti,
- La capacità di mantenere *consistenza* tra fasi di ingestione, predizione e persistenza.

Nel dominio medico i trade-off tra **precisione** e **sensibilità** ricoprono un ruolo critico. I falsi negativi possono portare a omissioni diagnostiche rilevanti, mentre i falsi positivi possono introdurre rumore operativo e aumentare il carico di verifica manuale.

Serve dunque una pipeline che produca classificazioni accurate e sia al contempo *interpretabile* dal punto di vista architetturale, scalabile e adatta a infrastrutture distribuite. In tale prospettiva, il progetto affronta il problema della classificazione di battiti ECG come un caso di studio per l'integrazione tra **machine learning classico** e **data streaming**.

Il contesto clinico rende particolarmente rilevante la distinzione tra *monitoraggio episodico* e *monitoraggio continuo*. Nel primo caso, l'ECG viene acquisito in ambulatorio o in reparto per intervalli limitati; nel secondo, dispositivi portatili o impiantati generano flussi prolungati che devono essere analizzati mentre arrivano. Le aritmie non sempre si manifestano durante una registrazione breve: ectopie isolate, pause o tachicardie parossistiche possono comparire a distanza di ore, rendendo insufficiente un'analisi puramente batch su archivi statici. Un sistema in grado

di classificare ogni battito al momento dell’acquisizione consente di segnalare tempestivamente pattern anomali, riducendo il ritardo tra evento fisiologico e possibile intervento clinico.

Dal punto di vista ingegneristico, il problema si colloca all’intersezione tra **signal processing**, **machine learning** e **ingegneria dei sistemi distribuiti**. La letteratura sulla classificazione automatica del MIT–BIH [7], [19] dimostra da decenni che modelli supervisionati su feature beat-level possono raggiungere accuratezze elevate; tuttavia, gran parte degli studi si concentra sulla fase di training e valutazione offline, senza integrare ingestion, messaggistica e persistenza in un’unica architettura operativa. Questa tesi colma, in parte, tale divario proponendo una pipeline che riproduce un ciclo di vita realistico del dato, dalla preparazione storica allo scoring in tempo quasi reale.

1.2 Approccio e scelte tecnologiche

In questo lavoro si affronta il problema della classificazione di battiti ECG in un contesto di streaming, privilegiando un’architettura che consenta **disaccoppiamento** tra ingestion e processing, resilienza ai guasti e scalabilità orizzontale.

La soluzione proposta impiega tecnologie consolidate nello stack Big Data:

- **Scala**: implementazione del sistema e dei componenti della pipeline,
- **Apache Kafka**: broker Pub/Sub per la messaggistica asincrona,
- **Apache Spark**: elaborazione batch (MLlib) e streaming (Structured Streaming),
- **Delta Lake**: sink transazionale per la persistenza dei risultati.

Questa combinazione consente di separare in modo netto la responsabilità di acquisizione dei dati dalla responsabilità di inferenza, evitando che il carico di una fase condizioni in maniera diretta l’altra.

La scelta di una soluzione *ibrida batch/streaming* risponde a un’esigenza precisa: i modelli predittivi possono essere addestrati su archivi storici con costi computazionali più elevati, mentre l’inferenza deve operare con latenza contenuta e comportamento deterministico. Il progetto adotta quindi una pipeline che riproduce un ciclo realistico di produzione dei dati, addestramento e serving, offrendo un contesto coerente per valutare non soltanto la qualità del classificatore, ma anche la solidità dell’infrastruttura sottostante.

Un’alternativa avrebbe potuto consistere in un’architettura monolitica: un unico processo che legge il CSV, addestra il modello e classifica i battiti in memoria senza broker intermedi. Tale soluzione sarebbe più semplice da implementare, ma non permetterebbe di valutare proprietà fondamentali di un sistema distribuito, come il disaccoppiamento tra velocità di produzione e consumo, la tolleranza ai guasti del consumer e la persistenza transazionale dei risultati. Analogamente, l’adozione di un framework di serving dedicato (TensorFlow Serving, MLflow Model Serving) avrebbe spostato il focus verso l’inferenza a bassa latenza, sacrificando l’integrazione nativa con Spark per training e streaming analitico. La scelta dello stack Kafka–Spark–Delta risponde invece a un obiettivo didattico e di ricerca: dimostrare come tecnologie consolidate nel panorama Big Data possano essere composte in una pipeline coerente per dati biomedici strutturati, senza dipendere da servizi cloud proprietari.

1.3 Obiettivi

Gli obiettivi perseguiti sono i seguenti:

1. Progettare e implementare una **pipeline end-to-end** per la classificazione di battiti ECG in near-real-time, rispettando requisiti di tolleranza ai fault, scalabilità e disaccoppiamento tra le componenti.
2. Sviluppare un processo di **addestramento batch** per un modello basato su Random Forest, includendo fasi di feature engineering, selezione delle variabili e validazione sperimentale.
3. Implementare un meccanismo di **ingestion realistico** tramite un simulatore (**EcgPatientSimulator**) che converte righe del CSV preprocessato (derivato dal MIT-BIH) in payload JSON e li pubblica su Kafka.
4. Eseguire **inferenza in streaming** con Spark Structured Streaming, garantendo atomicità delle scritture su Delta Lake tramite checkpointing e **foreachBatch**, così da preservare la continuità operativa in presenza di interruzioni.
5. Valutare la qualità del modello con metriche di classificazione e analizzare *latenze*, *throughput* e comportamento del sistema al variare del volume dei messaggi.

Gli obiettivi sono stati formulati in modo da coprire sia la dimensione *funzionale* (il sistema deve classificare correttamente i battiti) sia quella *non funzionale* (il sistema deve operare con latenza contenuta, recuperare da failure e restare comprensibile come architettura). La separazione tra training batch e inferenza streaming non è un vincolo imposto dal dataset, bensì una scelta progettuale che riflette la frequenza tipica di aggiornamento dei modelli in ambito clinico: il retraining avviene su scala di giorni o settimane, mentre l'inferenza deve procedere a ogni battito ricevuto. Analogamente, l'uso di un simulatore anziché di hardware reale consente di concentrare l'attenzione sull'integrazione software, lasciando aperte estensioni future verso dispositivi edge con firmware dedicato.

1.4 Contributi

Si elencano i contributi principali del lavoro:

- **Realizzazione di una pipeline sperimentale completa** in Scala che integra training batch e inferenza in streaming su dati ECG simulati, mantenendo la separazione tra livelli di acquisizione, elaborazione e persistenza.
- **Approccio pragmatico al feature engineering** su battiti cardiaci (intervalli pre-RR/post-RR, descrittori morfologici del QRS, durate PQ/QT/ST e coefficienti **qrs_morph** su due derivazioni) e definizione di uno schema JSON compatto per l'ingestion in Kafka.
- Dimostrazione dell'integrazione di Spark Structured Streaming con schema dinamico (**from_json**), checkpointing e scritture transazionali su Delta Lake per garantire resilienza, consistenza e ripetibilità delle elaborazioni.

- Analisi sperimentale completa della performance del modello (tramite `MulticlassClassificationEvaluator`) e delle proprietà non funzionali della pipeline, con attenzione a latenze, throughput e stabilità operativa.
- Formalizzazione di un caso di studio in cui una stessa architettura può essere riletta sia come sistema di prototipazione accademica sia come base per un futuro deployment in ambito e-Health.

Rispetto a implementazioni isolate di classificatori ECG presenti in letteratura, il contributo distintivo risiede nell'**integrazione end-to-end** piuttosto che nella ricerca del massimo score su un benchmark. Il lavoro documenta non solo il modello predittivo, ma anche le scelte di configurazione (checkpoint, `foreachBatch`, schema JSON dinamico), i trade-off architetturali (Lambda ibrida, mono-nodo vs cluster) e le implicazioni cliniche degli errori di classificazione. Questa impostazione rende la tesi utile come riferimento per chi deve progettare pipeline analitiche in ambito sanitario, dove la correttezza del flusso dati è tanto importante quanto l'accuratezza del classificatore.

1.5 Struttura della tesi

La trattazione è organizzata come segue:

- Il **Capitolo 2** descrive il dominio applicativo e il dataset utilizzato (MIT-BIH in versione tabellare da Kaggle), includendo considerazioni su feature cliniche, preprocessing del segnale e paradigma Edge Computing.
- Il **Capitolo 3** giustifica le scelte tecnologiche adottate, soffermandosi sui vantaggi di Scala, Spark, Kafka KRaft e Delta Lake nel contesto di pipeline distribuite.
- Il **Capitolo 4** illustra l'architettura complessiva e la scomposizione in tre moduli principali: `EcgModelTrainer` (batch), `EcgPatientSimulator` (ingestion) e `EcgStreamingPredictor` (streaming).
- Il **Capitolo 5** approfondisce dettagli implementativi, mostrando frammenti di codice e spiegando le scelte di configurazione della ML pipeline e del producer Kafka.
- Il **Capitolo 6** presenta la valutazione sperimentale, confrontando metriche di accuratezza, analisi di falsi positivi/negativi, e misure di latenze e throughput.
- Il **Capitolo 7** espone le valutazioni complessive e le possibili linee di estensione verso modelli deep e integrazione analitica con Delta Lake.

I sorgenti del repository (`EcgModelTrainer`, `EcgPatientSimulator`, `EcgStreamingPredictor`) sono richiamati nei capitoli di implementazione e risultati, così da legare teoria e codice effettivamente scritto.

La progressione argomentativa segue un percorso dal generale al particolare: il Capitolo 2 fonda il problema nel dominio clinico e nel dataset scelto; il Capitolo 3 giustifica le tecnologie; i Capitoli 4 e 5 descrivono rispettivamente la progettazione logica e i dettagli implementativi; il Capitolo 6 chiude il ciclo con la valutazione sperimentale. Dove possibile, ogni scelta tecnologica è collegata a un requisito esplicito, in modo che il lettore possa ricostruire il ragionamento progettuale senza dover inferire le motivazioni dalle sole configurazioni riportate nel codice.

Capitolo 2

Dominio Applicativo e Dataset

2.1 Dominio applicativo: il segnale ECG

Il segnale elettrocardiografico (ECG) rappresenta la proiezione elettrica dell'attività cardiaca misurata sulla superficie corporea. Dal punto di vista fisiologico, esso riflette la sequenza di depolarizzazione e ripolarizzazione delle camere cardiache e consente di osservare, con risoluzione temporale elevata, eventi che sono spesso immediatamente correlati allo stato elettrico del miocardio [16].

2.1.1 Elettrofisiologia e morfologia del battito

L'attivazione elettrica del cuore origina nel **nodo seno-atriale** (SA), che innesca la depolarizzazione degli atri. Il fronte di onda raggiunge poi i ventricoli attraverso il **nodo atrioventricolare** (AV), il **fascio di His** e il sistema delle **fibre di Purkinje**, che distribuiscono l'impulso sul miocardio ventricolare in modo coordinato. La sequenza temporale di queste fasi si traduce, sul tracciato, in segmenti e intervalli con significato clinico ben definito.

La morfologia tipica di un battito in ritmo sinusale è costituita dalle onde **P**, **QRS** e **T**: l'onda P corrisponde alla depolarizzazione atriale; il complesso QRS alla depolarizzazione ventricolare (con i deflessi Q, R e S che descrivono l'attivazione sequenziale del setto e delle pareti ventricolari); l'onda T alla ripolarizzazione ventricolare. Tra questi deflessi si misurano l'**intervallo PR** (conduzione atrioventricolare), il **segmento ST** (fase di ripolarizzazione precoce ventricolare) e l'**intervallo QT** (durata dell'attività ventricolare totale), spesso corretto per la frequenza cardiaca (QTc) nelle valutazioni cliniche.

La Figura 2.1 sintetizza, in forma schematica, la relazione tra fasi elettriche e morfologia osservabile. In applicazioni automatiche, la forma locale del QRS e la durata degli intervalli costituiscono descrittori ad alta densità informativa per la distinzione tra battiti normali e patologici [7].

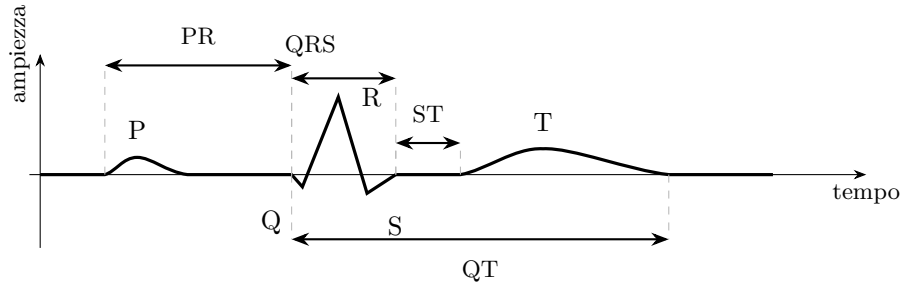


Figura 2.1: Morfologia schematica di un battito ECG in ritmo sinusale: deflessioni P, Q, R, S e T; intervallo PR, durata del complesso QRS, segmento ST e intervallo QT.

Gli *intervalli RR* (distanza temporale tra picchi R consecutivi) sono la misura principale per la frequenza cardiaca e per la *Heart Rate Variability* (HRV). In presenza di aritmie, la distribuzione degli RR perde regolarità: ectopie, pause o tachicardie producono sequenze che un classificatore può sfruttare insieme alla morfologia del battito isolato [7], [26].

Dal punto di vista diagnostico, le alterazioni morfologiche e ritmiche non sono equivalenti. Un battito con morfologia QRS normale ma intervallo RR anomalo (ad esempio una extrasistole con compensazione incompleta) può essere classificato diversamente da un battito con QRS allargato e morfologia bizzarra tipica di un'origine ventricolare. Per questo motivo i descrittori tabellari del dataset impiegato combinano entrambe le prospettive: intervalli **pre-RR** e **post-RR** per il contesto ritmico, ampiezze e durate per la morfologia istantanea. La classificazione automatica, in questo contesto, non mira a sostituire l'interpretazione clinica complessiva del tracciato, ma a supportare lo screening di singoli battiti in flussi ad alta frequenza, dove la revisione manuale di ogni evento sarebbe impraticabile.

2.1.2 Variabilità della frequenza cardiaca

La HRV quantifica le fluttuazioni dell'intervallo RR attorno al valore medio e riflette, in parte, l'equilibrio tra i sistemi simpatico e parasimpatico. Le linee guida del Task Force del 1996 [26] distinguono due famiglie di misure, entrambe rilevanti per il feature engineering adottato in questo lavoro.

Nel **dominio del tempo**, le metriche più diffuse includono:

- **SDNN**: deviazione standard degli intervalli NN (battiti di origine sinusale);
- **RMSSD**: radice quadrata della media dei quadrati delle differenze successive tra intervalli NN;
- **pNN50**: percentuale di coppie di intervalli NN consecutivi la cui differenza supera 50 ms.

RMSSD e pNN50 sono particolarmente sensibili alle componenti ad alta frequenza, associate all'attività vagale a breve termine.

Nel **dominio della frequenza**, lo spettro della serie RR (tipicamente ricampionata e detrendata) viene suddiviso in bande: la componente **LF** (circa 0.04–0.15 Hz) e la componente

HF (circa 0.15–0.40 Hz), quest’ultima legata alla respirazione e al tono parasimpatico. Il **rapporto LF/HF** è usato in letteratura come indicatore sintetico dell’equilibrio autonomico, sebbene la sua interpretazione clinica richieda cautela e contesto del paziente [26].

Nel dataset impiegato, metriche HRV globali (SDNN, RMSSD, spettro LF/HF) non sono calcolate in tempo reale dalla pipeline Scala; il CSV contiene però intervalli RR locali (**pre-RR**, **post-RR**) e descrittori morfologici già estratti offline dal segnale grezzo. Il classificatore opera quindi su un livello di astrazione che unifica morfologia istantanea e contesto ritmico immediato attorno al battito corrente.

In letteratura clinica, la HRV globale è utilizzata per valutare il rischio cardiaco, lo stress autonomico e la risposta a interventi terapeutici. Nel contesto della classificazione beat-to-beat, invece, le metriche globali (SDNN, spettro LF/HF) sono meno direttamente applicabili: ogni battito viene classificato in isolamento o con un contesto ristretto agli intervalli RR adiacenti. Questa distinzione è importante per interpretare correttamente i risultati del modello: un’elevata accuratezza sulla classe N non implica necessariamente una buona caratterizzazione della variabilità ritmica complessiva del paziente, bensì la capacità di riconoscere singoli battiti la cui morfologia e contesto RR locale sono coerenti con il ritmo sinusale.

2.1.3 Rappresentazione computazionale e feature

Dal punto di vista computazionale, il battito ECG può essere rappresentato come una **sequenza multiscala**: una componente temporale (distanza tra eventi) e una componente morfologica (forma del tracciato locale). Questa doppia prospettiva consente di trasformare un segnale continuo in un insieme di osservazioni etichettate, ciascuna associata a un istante clinicamente significativo, adatto a modelli supervisionati che richiedono vettori numerici stabili nel tempo.

Nel file MIT-BIH `Arrhythmia Database.csv` del corpus Kaggle [23], [24], ogni riga descrive un singolo battito con **32 feature numeriche** (16 per ciascuna delle due derivazioni, prefissi `0_` per lead II e `1_` per lead V5), oltre a **record** (identificativo registrazione) e **type** (etichetta):

- **Contesto ritmico:** `pre-RR` e `post-RR` (intervalli R–R rispettivamente precedente e successivo al battito corrente, in ms) [7].
- **Morfologia delle onde:** ampiezze `pPeak`, `qPeak`, `rPeak`, `sPeak`, `tPeak`.
- **Durata degli intervalli:** `pq_interval`, `qrs_interval`, `st_interval`, `qt_interval`.
- **Forma del QRS:** cinque coefficienti `qrs_morph0`–`qrs_morph4` che riassumono la morfologia del complesso [8].

Tali attributi costituiscono il *vettore di ingresso* per modelli di machine learning classico (Random Forest nel caso di studio) e sono prodotti da una catena di preprocessing che precede l’addestramento batch. La scelta di non trasmettere il segnale raw riduce la complessità del modello, limita il volume dei messaggi Kafka e rende esplicita la semantica di ogni campo nel payload JSON.

La duplicazione delle feature su due derivazioni (lead II e V5) riflette una pratica consolidata in cardiologia: ciascuna derivazione offre una prospettiva diversa sull’attività elettrica ventricolare, e la combinazione di più canali può migliorare la discriminazione tra classi con morfologie simili

su un singolo lead. Nel CSV preprocessato, i prefissi `0_` e `1_` mantengono questa separazione senza richiedere algoritmi di fusione espliciti: il `VectorAssembler` concatena tutte le colonne numeriche in un unico vettore, lasciando al Random Forest il compito di apprendere le interazioni tra derivazioni. Questo approccio è pragmatico per un prototipo, sebbene in un sistema maturo si potrebbero esplorare tecniche di feature selection o modelli multi-input che pesano diversamente ciascuna derivazione in funzione della qualità del segnale.

2.1.4 Preprocessing e qualità del segnale

La qualità del segnale condiziona in modo diretto l'affidabilità delle feature. Prima della segmentazione beat-to-beat, la letteratura propone una sequenza di operazioni che, nel nostro flusso, sono applicate offline sulla fase di costruzione del CSV, ma che in un deployment edge replicherebbero lo stesso ordine logico [8], [22].

Filtraggio. Un **filtro passa-alto** (tipicamente con frequenza di taglio intorno a 0.5 Hz) attenua il *baseline wander*, derivante da respirazione, movimento del paziente o deriva degli elettrodi. Un **filtro notch** centrato a 50 Hz o 60 Hz rimuove l'interferenza della rete elettrica (*power-line interference*). Un **filtro passa-basso** (spesso tra 35 e 40 Hz, coerente con la banda utile del MIT-BIH) limita il rumore muscolare e le componenti ad alta frequenza poco informative per la classificazione dei battiti. In pratica, la combinazione di questi stadi equivale a un filtraggio passa-banda nell'intervallo 0.5–40 Hz citato nelle specifiche del database.

Rilevamento dei picchi R. L'algoritmo di *Pan-Tompkins* [22] è un riferimento consolidato per la detection in tempo quasi reale. La pipeline prevede: (i) filtraggio passa-banda; (ii) derivata per enfatizzare le pendenze del QRS; (iii) elevamento al quadrato per uniformare la polarità; (iv) integrazione a finestra mobile per smussare il segnale; (v) soglia adattiva su due livelli per distinguere picchi veri da artefatti. La robustezza dipende dalla calibrazione delle soglie e dalla qualità del tracciato: in presenza di aritmie con morfologie atipiche, un detector generico può richiedere post-processing o regole cliniche aggiuntive.

Indice di qualità del segnale. Gli artefatti più frequenti in acquisizioni ambulatoriali includono interferenza elettromagnetica, contatto instabile degli elettrodi, movimenti del paziente e saturazione dell'amplificatore. Ciascuno di questi fenomeni può distorcere le ampiezze delle onde o introdurre picchi spuri che un detector QRS interpreta come battiti aggiuntivi. Per evitare che tali artefatti degradino le feature, si adottano *Signal Quality Index* (SQI) che combinano metriche nel dominio del tempo, della frequenza e della forma d'onda [15]. Battiti associati a tratti di segnale con SQI insufficiente possono essere scartati, marcati come *unknown* o inviati su un canale di quarantena, riducendo la propagazione di errori verso il classificatore. In una pipeline distribuita, questa valutazione ha anche valenza architetturale: filtrare a monte significa trasmettere verso il broker solo eventi affidabili e alleggerire il carico del livello streaming. Nel flusso sperimentale di questa tesi, la qualità del segnale è implicitamente garantita dal preprocessing offline che ha prodotto il CSV Kaggle; in un deployment reale, la valutazione SQI andrebbe integrata nel percorso edge prima della pubblicazione su Kafka.

Il preprocessing può dunque essere letto come **normalizzazione semantica**: il segnale fisiologico diventa una rappresentazione tabellare allineata al modello ML. In produzione, la fase

può risiedere sul dispositivo wearable o su un gateway, così da inviare al data center feature già validate anziché campioni raw ad alta frequenza.

2.2 Dataset sperimentale: MIT–BIH tabellare (Kaggle)

Per la fase sperimentale si utilizza la versione tabellare del **MIT–BIH Arrhythmia Database** pubblicata nel corpus Kaggle *ECG Arrhythmia Classification Dataset* [23], [24], derivato dalla raccolta originale su PhysioNet [11], [19]. Il bundle Kaggle contiene più file CSV (MIT–BIH, MIT–BIH Supraventricular, INCART, SCD Holter); il prototipo Scala legge tramite `DATASET_PATH` i file `*.csv` presenti nella directory configurata: nelle sperimentazioni descritte in questa tesi si assume il file `MIT–BIH Arrhythmia Database.csv`, così da restare allineati al benchmark MIT–BIH senza mescolare registrazioni di altri database.

2.2.1 Origine e caratteristiche del corpus PhysioNet

Il MIT–BIH originale [11], [19] è una raccolta acquisita tra il 1975 e il 1979 presso il Beth Israel Hospital (oggi Beth Israel Deaconess Medical Center), con campionamento a 360 Hz e risoluzione di 11 bit su circa 10 mV di dinamica. Il corpus è diventato, nel tempo, uno *benchmark* de facto per la classificazione automatica dei battiti.

Le proprietà salienti del database grezzo sono le seguenti:

- 48 registrazioni di circa 30 minuti ciascuna, da 47 soggetti (due tracce per un paziente).
- Due canali per registrazione, derivazioni differenti (tipicamente lead I e una derivazione del lead II modificato).
- Annotazioni beat-to-beat curate da più esperti, con codici simbolici per oltre venti categorie di battiti ed eventi non ritmici.

La versione CSV impiegata nel progetto non contiene campioni raw del segnale, bensì battiti già segmentati con feature estratte offline (filtraggio, rilevamento QRS, calcolo di intervalli e descrittori morfologici) [7], [24]: la pipeline Scala legge questi archivi senza ripetere sul cluster l'intera catena descritta in §2.1.4.

Il corpus PhysioNet [11] ha svolto un ruolo fondamentale nella standardizzazione della ricerca su segnali fisiologici, offrendo accesso libero a database annotati e toolkit di analisi. Il MIT–BIH Arrhythmia Database, in particolare, è stato utilizzato in centinaia di pubblicazioni come benchmark per confrontare algoritmi di classificazione, rilevamento QRS e misura di intervalli. La sua longevità comporta però anche limiti noti: acquisizioni datate, popolazione non rappresentativa dell'attuale demografia clinica, e annotazioni che, sebbene curate da esperti, non sono immune da disaccordi inter-osservatore su casi borderline. Per una tesi focalizzata sull'architettura della pipeline, questi limiti sono accettabili; per una validazione clinica, richiederebbero integrazione con dataset più recenti e diversificati.

2.2.2 Schema del file CSV e volume dati

Il file MIT-BIH Arrhythmia Database.csv contiene **100 689** battiti da **44** registrazioni (colonna **record**), con **34** colonne totali (identificativo, etichetta e 32 feature). Quattro tracce presenti nel MIT-BIH grezzo non compaiono nel CSV preprocessato, probabilmente escluse in fase di estrazione per qualità del segnale o annotazioni incomplete. Ogni riga corrisponde a un battito; le feature duplicano la stessa semantica su due derivazioni (lead II, prefisso 0_; lead V5, prefisso 1_), come riassunto in Tabella 2.3.

Il volume di circa 100 000 battiti è sufficiente per addestrare modelli ensemble su singola macchina con tempi di esecuzione contenuti, ma non rappresenta la scala di un deployment multi-paziente in cui milioni di battiti al giorno confluiscono verso un data center. In tale prospettiva, la pipeline descritta in questa tesi funge da *prova di concetto* su scala ridotta: le stesse tecnologie (Kafka, Spark, Delta) sono progettate per gestire volumi ordini di grandezza superiori, come discusso nei capitoli dedicati allo stack tecnologico e alla scalabilità. La scelta di limitare il simulatore a 1000 messaggi per run sperimentale riduce ulteriormente il carico, mantenendo comunque un percorso end-to-end verificabile.

2.2.3 Schema di raggruppamento AAMI e etichette nel CSV

Per confrontare i risultati con la letteratura clinica e con lo standard AAMI EC57:2012 [5], le annotazioni originali del MIT-BIH vengono raggruppate in cinque **super-classi**, come proposto da de Chazal et al. [7] e adottato nel Capitolo 6. Nel CSV Kaggle, la colonna **type** adotta una nomenclatura leggermente diversa dalle sigle AAMI letterali, ma con corrispondenza biunivoca (Tabella 2.1).

Tabella 2.1: Corrispondenza tra etichette nel CSV Kaggle e super-classi AAMI.

CSV (type)	AAMI	Significato
N	N	Normale
SVEB	S	Ectopia supraventricolare
VEB	V	Ectopia ventricolare
F	F	Fusione
Q	Q	Non classificabile

Tabella 2.2: Raggruppamento AAMI delle annotazioni MIT-BIH (sintesi del contenuto clinico).

Classe	Sigla	Contenuto tipico
N	Normale	Battiti sinusali, blocchi di branca sinistro (L), blocchi di branca destro (R)
S	Supraventricolare	Ectopie atriali (A), aberranti (a), nodali (J), ritmi supraventricolari
V	Ventricolare	Contrazioni premature ventricolari (V), ectopie ventricolari (E)
F	Fusion	Battiti di fusione (F)
Q	Unknown	Non classificabili, rumore marcato (/), altre etichette non mappate

La Tabella 2.2 non esaurisce ogni codice del database, ma fissa il vocabolario con cui si discutono metriche e errori clinici nel resto della tesi. Lo `StringIndexer` di Spark ML indicizza i valori testuali effettivi (N, SVEB, VEB, F, Q) senza rinomina esplicita verso le sigle AAMI: la mappatura resta a carico dell’analisi dei risultati.

Tabella 2.3: Feature per derivazione nel CSV MIT-BIH (prefissi 0_ e 1_).

Suffisso colonna	Contenuto
pre-RR, post-RR	Intervalli R-R precedente e successivo (ms)
pPeak ... tPeak	Ampiezze onde P, Q, R, S, T
pq_interval, ..., qt_interval	Durate PQ, QRS, ST, QT (ms)
qrs_morph0-qrs_morph4	Descrittori morfologici del complesso QRS

2.2.4 Sbilanciamento tra classi

La distribuzione dei battiti nel file MIT-BIH `Arrhythmia Database.csv` è fortemente *sbilanciata* (conteggi assoluti sul corpus completo): N 90 083 (89,5%), VEB 7 009 (7,0%), SVEB 2 779 (2,8%), F 803 (0,8%), Q 15 (<0,1%). Tale proporzione riflette la prevalenza clinica dei ritmi normali in registrazioni ambulatoriali selezionate, ma penalizza i modelli che ottimizzano l’accuratezza globale senza pesi per classe.

Durante l’addestramento si possono considerare **class weighting**, undersampling della classe maggioritaria o oversampling sintetico delle minoritarie; ciascuna strategia introduce trade-off (perdita di informazione, rischio di overfitting sulle classi rare, tempi di training più lunghi). Nel prototipo attuale non è implementato alcun ribilanciamento esplicito: la scelta è coerente con l’obiettivo di dimostrare l’architettura end-to-end, ma va tenuta presente nell’interpretazione delle metriche del Capitolo 6.

Dal punto di vista sperimentale, il MIT-BIH offre un compromesso favorevole tra complessità e riproducibilità: registrazioni sufficientemente eterogenee per stressare la pipeline, ma di dimensione gestibile in un contesto accademico in cui l’obiettivo principale non è il record di performance isolato, bensì la validazione di un flusso integrato dal dato storico allo streaming.

Le tecniche di ribilanciamento meritano una discussione più approfondita. L’**undersampling** della classe maggioritaria riduce i tempi di training e bilancia le classi, ma scarta informazione potenzialmente utile sui pattern del ritmo normale. L’**oversampling** (random o sintetico, ad esempio SMOTE) aumenta la rappresentazione delle classi rare, ma rischia di generare esempi artificiali non rappresentativi della variabilità clinica reale. Il **class weighting** penalizza maggiormente gli errori sulle classi minoritarie durante l’addestramento, senza alterare la distribuzione dei dati; Spark MLlib supporta pesi per classe in diversi classificatori, sebbene non sia stato attivato nel prototipo. La scelta di non applicare ribilanciamento esplicito rende i risultati più conservativi e più facili da confrontare con studi che adottano la stessa impostazione, ma va tenuta presente nell’interpretazione delle metriche sulle classi F e Q.

2.2.5 Tipologie di anomalie contenute e limiti

Le annotazioni includono ectopie ventricolari (PVC), alterazioni di conduzione, battiti supraventricolari e, in alcune tracce, eventi rari o tratti rumorosi che mettono alla prova la segmentazione. Il database non copre tutte le condizioni cliniche: aritmie prolungate come la fibrillazione atriale persistente o pattern legati a insufficienza cardiaca avanzata possono essere sottorappresentati.

La *generalizzabilità* verso altre popolazioni (età, sesso, patologia, dispositivo di acquisizione) non va assunta per default. I pazienti del MIT-BIH sono in gran parte adulti ricoverati o in follow-up cardiologico negli Stati Uniti della fine degli anni Settanta; un modello addestrato solo su questa distribuzione può degradare su cohort europee, pediatriche o su segnali da wearable consumer con banda e rumore differenti.

Per un classificatore supervisionato, la coerenza tra annotazioni, segmentazione e feature è tanto importante quanto l'algoritmo di apprendimento: errori a monte si propagano come etichette rumorose o descrittori distorti, limitando il soffitto di performance raggiungibile anche con modelli più complessi.

Tra le anomalie più frequenti nel MIT-BIH figurano le **contrazioni premature ventricolari** (PVC), caratterizzate da QRS largo e morfologia distinta dal ritmo sinusale, e le **extrasistoli supraventricolari**, spesso con QRS stretto ma intervallo RR anomalo. I **battiti di fusione** (classe F) rappresentano un caso limite in cui un impulso ventricolare e uno sinusale si sovrappongono temporalmente, producendo morfologie ibride difficili da classificare anche per annotatori umani. La classe Q (*unknown*) raggruppa battiti non classificabili, tratti rumorosi o etichette non mappate nello schema AAMI: la sua rarissima presenza nel CSV ($<0,1\%$) rende quasi impossibile una valutazione statistica affidabile su questa categoria.

2.3 Edge Computing e ruolo del simulatore

Nel monitoraggio cardiaco continuo, la posizione dell'elaborazione lungo il percorso dato-cloud non è banale. Si distinguono almeno tre livelli [12]:

- **Cloud:** capacità computazionale elevata, storage centralizzato, latenza di rete non trascurabile;
- **Fog:** nodi intermedi (ospedale, edge data center) che aggregano flussi da più dispositivi;
- **Edge:** elaborazione sul wearable, sul Holter o sul gateway domestico, a ridosso del sensore.

Per l'ECG, spostare filtraggio, QRS detection e estrazione delle feature verso l'edge risponde a tre esigenze concrete: ridurre la latenza tra evento cardiaco e allarme; diminuire il traffico (un battito con 32 feature numeriche occupa ordini di grandezza in meno di un secondo di segnale a 360 Hz); limitare l'esposizione di dati biometrici raw oltre il perimetro controllato del paziente.

Un confronto quantitativo illustra il vantaggio in termini di banda. Un secondo di segnale ECG a 360 Hz con risoluzione a 16 bit su un canale produce circa 720 byte di campioni raw; lo stesso battito, rappresentato come JSON con 32 feature numeriche e metadati, occupa tipicamente qualche centinaio di byte. Moltiplicando per una frequenza cardiaca media di 60–80 battiti

al minuto, il risparmio cumulato su ore di monitoraggio diventa significativo, soprattutto su connessioni mobili con costi di trasferimento o limiti di batteria sul dispositivo wearable. Inoltre, elaborare localmente consente di applicare filtri e SQI prima della trasmissione, evitando di saturare il broker con eventi di bassa qualità.

Dispositivi come Holter multicanale, patch ECG monouso e braccialetti con singola derivazione implementano già, in parte, questa logica: acquisiscono a alta frequenza localmente, poi trasmettono riassunti, eventi o tratti selezionati. Il progetto di tesi non dispone di hardware dedicato; il componente `EcgPatientSimulator` emula il comportamento di un producer edge che pubblica feature già calcolate sul topic Kafka `ecg-input-stream`, in formato JSON.

2.3.1 Funzione metodologica del simulatore

Il simulatore consente di esercitare la pipeline end-to-end in laboratorio: stesso schema colonne del CSV di training, stesso topic, stesso consumer Spark, senza dipendenze da firmware o protocolli proprietari. In uno scenario operativo, lo sostituirebbe un modulo on-device (o un gateway) con `KafkaProducer` o equivalente MQTT verso il broker.

L’approccio edge comporta vincoli noti: CPU e memoria limitate, consumo energetico, necessità di aggiornare modelli e soglie senza interrompere l’acquisizione. Per la tesi, questi aspetti restano fuori dall’implementazione, ma orientano le scelte architetturali del Capitolo 4 (payload compatto, disaccoppiamento tramite Kafka, training batch separato dall’inferenza streaming).

La riproducibilità degli esperimenti ne beneficia: la sequenza di messaggi può essere rigenerata con la stessa logica (`limit(1000)`, intervallo di un secondo), permettendo confronti tra configurazioni del predictor o del checkpoint senza variabilità legata a dispositivi reali.

Dal punto di vista architetturale, il simulatore occupa il ruolo di *producer* nel diagramma logico della pipeline: non partecipa all’addestramento né alla persistenza, ma alimenta il topic Kafka con lo stesso formato che un dispositivo edge produrrebbe in produzione. Questa separazione è deliberata e consente di sostituire il simulatore con un producer reale senza modificare il predictor o il trainer. In uno scenario multi-paziente, più istanze del simulatore (o di dispositivi reali) potrebbero pubblicare sullo stesso topic, sfruttando le partizioni Kafka per parallelizzare il consumo; nel prototipo attuale, un singolo producer sequenziale è sufficiente per dimostrare il flusso.

2.3.2 Scenario evolutivo: digital twin

A orizzonte più ampio, i flussi ECG da molteplici fonti potrebbero alimentare un *digital twin* cardiaco del paziente: modello che integra storico clinico, eventi in tempo quasi reale e predizioni per simulare risposte a terapie o per prioritizzare alert. Il prototipo descritto in questa tesi costituisce un mattone elementare di tale visione—ingestion, classificazione, persistenza versionata—senza pretendere di realizzarne la complessità clinica e regolatoria.

Un *digital twin* cardiaco richiederebbe l’integrazione di molteplici fonti dati oltre all’ECG: pressione arteriosa, imaging, farmaci assunti, storia clinica del paziente. La pipeline implementata fornisce il sottosistema di **ingestion e scoring in tempo quasi reale** che alimenterrebbe tale

modello con eventi classificati e timestampati. Delta Lake, con il supporto al *time travel*, consentirebbe di ricostruire lo stato delle predizioni a un istante passato, utile per audit clinici o per simulare l'effetto di diverse soglie di allarme su dati storici. La persistenza versionata delle predizioni è un prerequisito per qualsiasi sistema che debba giustificare retrospettivamente una decisione clinica supportata da algoritmo.

2.4 Considerazioni etiche e implicazioni cliniche

Nell'applicazione di modelli ML a dati clinici, l'impatto di **falsi negativi** e **falsi positivi** va valutato prima delle metriche aggregate. Un falso negativo su una ectopia ventricolare ripetuta può ritardare un intervento; un falso positivo aumenta il carico di revisione per il personale e, nel lungo periodo, riduce la fiducia nel sistema di alert.

Il problema è dunque operativo oltre che statistico: output, latenza e priorità degli allarmi devono essere allineati al contesto (telemetria domiciliare, reparto, terapia intensiva), dove i trade-off tra sensibilità e specificità non coincidono.

Quadro regolatorio. In Europa, software destinato a finalità mediche può rientrare nel campo del Regolamento (UE) 2017/745 sui dispositivi medici (MDR) [10]. I sistemi di supporto alla diagnosi basati su algoritmi—spesso indicati come *Software as a Medical Device* (SaMD)—richiedono gestione del rischio, tracciabilità, validazione clinica e monitoraggio post-market. Il prototipo di questa tesi non è certificato né destinato all'uso clinico; le considerazioni regolatorie servono a inquadrare il divario tra dimostrazione accademica e prodotto deployabile.

Validazione, interpretabilità e bias. Un'adozione responsabile presuppone validazione su popolazioni e dispositivi diversi da quelli del training, documentazione delle prestazioni per sottogruppo e strumenti di interpretabilità (importanza delle feature, analisi degli errori per classe AAMI). Il MIT-BIH, per quanto prezioso, introduce *bias* demografico e tecnologico: campione storico, non necessariamente rappresentativo dell'utenza attuale né delle acquisizioni da wearable.

Infine, la pipeline deve essere leggibile da chi risponde dei dati: finalità del trattamento, minimizzazione (feature anziché raw dove possibile), retention su Delta Lake e accessi al broker vanno progettati con pari attenzione rispetto all'accuratezza del classificatore. Senza questi accorgimenti, anche un modello accurato resterebbe confinato al laboratorio.

Protezione dei dati e minimizzazione. Il Regolamento Generale sulla Protezione dei Dati (GDPR) impone principi di liceità, trasparenza, minimizzazione e limitazione della conservazione per i dati sanitari. La pipeline adotta, in parte, il principio di minimizzazione trasmettendo feature estratte anziché segnali raw, riducendo la quantità di informazione biometrica sensibile che transita sul broker. Tuttavia, anche le feature morfologiche e gli identificativi di registrazione (**record**) possono consentire la re-identificazione in contesti specifici; un deployment reale richiederebbe pseudonimizzazione degli identificativi, cifratura end-to-end e policy di retention definite contrattualmente con il titolare del trattamento.

Equità e rappresentatività. I modelli di machine learning possono ereditare e amplificare bias presenti nei dati di training. Il MIT-BIH, acquisito quasi cinquant'anni fa su una popolazione ospedaliera statunitense, potrebbe non generalizzare equamente su altre etnie, fasce d'età o condizioni comorbide. Un sistema deployato senza validazione su sottogruppi rischia di performare in modo diseguale, con implicazioni etiche oltre che cliniche. La documentazione trasparente delle performance per classe AAMI, come proposta nel Capitolo 6, è un primo passo verso una valutazione responsabile, ma non sostituisce studi di validazione esterna su popolazioni diverse.

Capitolo 3

Lo Stack Tecnologico

3.1 Il linguaggio Scala: programmazione funzionale applicata ai Big Data

Scala è un linguaggio multi-paradigma che combina programmazione orientata agli oggetti e programmazione funzionale sulla Java Virtual Machine (JVM). La sua progettazione punta a un *type system* espressivo, a immutabilità per default nelle strutture dati e a interoperabilità diretta con l'ecosistema Java, caratteristiche rilevanti per sistemi distribuiti di medie e grandi dimensioni [21].

Nel contesto di questo lavoro, Scala non è stato scelto per preferenze sintattiche, bensì perché rappresenta il *linguaggio nativo* di Apache Spark. Le API di Spark sono scritte in Scala; le trasformazioni su `Dataset` tipizzati sfruttano *encoder* derivati dal compilatore, evitando la serializzazione Python-JVM di PySpark, che introduce overhead non trascurabile quando il volume di eventi cresce [28].

Il progetto adotta la versione **2.13.18** (`build.sbt`), con prefisso di package `it.utiu.thesis`, in linea con Spark 4.1.1 e con le ottimizzazioni delle collezioni della serie 2.13.

La JVM offre inoltre un ecosistema maturo di librerie Java interoperabili: il simulatore utilizza direttamente `KafkaProducer` del client Apache Kafka, senza wrapper aggiuntivi. Questa interoperabilità è rilevante in contesti eterogenei dove alcuni componenti (sensori, gateway, servizi legacy) sono implementati in Java e devono integrarsi con job Spark in Scala. La gestione delle dipendenze tramite SBT risolve le versioni di Spark, Delta e Kafka in modo dichiarativo, riducendo i conflitti di classpath che spesso emergono in progetti Big Data multi-modulo.

3.1.1 Immutabilità, composizione e type system

Un principio ricorrente nel codice è l'uso di trasformazioni *lazy* su `DataFrame` anziché mutazioni esplicite. In `EcgModelTrainer`, la catena `read` $\rightarrow$ `na.drop` $\rightarrow$ `randomSplit` costruisce nuove viste logiche senza modificare il dataset precedente; la pipeline ML concatena `StringIndexer`, `VectorAssembler` e `RandomForestClassifier` in un unico `Pipeline` immutabile.

Le **case class** e il **pattern matching** facilitano la modellazione di strutture dati e la gestione di casi limite (ad esempio, assenza del modello su disco in **AppRunner**) con esaustività verificata dal compilatore. Nel dominio Spark, il vantaggio del type system emerge soprattutto con **Dataset[T]**: gli *implicit encoder* permettono serializzazione efficiente di tipi custom, a differenza di PySpark dove gran parte del percorso resta reflection-based e più costoso su piccoli batch ad alta frequenza.

In un sistema con thread concorrenti (predictor in background, simulatore sul main thread), l'assenza di stato condiviso mutabile tra componenti riduce race condition e semplifica il ragionamento sul comportamento in failure.

Il pattern matching di Scala si rivela utile anche nella gestione del ciclo di vita dell'applicazione. In **AppRunner**, la verifica dell'esistenza del modello su disco può essere espressa in modo dichiarativo, e il ramo di esecuzione che avvia il trainer è separato da quello che procede direttamente all'inferenza. Questa chiarezza strutturale riduce il rischio di errori logici rispetto a catene di **if-else** annidati, soprattutto quando il numero di componenti orchestrati cresce.

3.1.2 Confronto con alternative

PySpark offre una curva di apprendimento più rapida, ma ogni invocazione attraversa Py4J. Con un messaggio al secondo il costo è trascurabile; con centinaia di battiti al minuto per paziente e più stream paralleli, la latenza di bridging diventa misurabile. Java puro manca della concisione per esprimere pipeline dichiarative in poche righe. Scala resta, per questo progetto, il compromesso preferito: runtime JVM, API Spark native, codice batch e streaming nello stesso repository.

Linguaggi come Python e R restano eccellenti per l'esplorazione dati e la prototipazione rapida di modelli, ma introducono un *impedance mismatch* quando il deployment richiede integrazione nativa con Spark Streaming e Delta Lake su JVM. La scelta di Scala allinea il linguaggio di implementazione al runtime di esecuzione, eliminando il bridging Py4J e consentendo di condividere tipi e serializzatori tra training e inferenza senza conversioni intermedie.

3.2 Apache Spark: Core, Spark SQL e MLlib

Apache Spark generalizza MapReduce con operazioni in-memory su strutture resilienti (RDD e, oggi, tabelle SQL) [27], [28]. Il progetto utilizza **Spark 4.1.1** con moduli **spark-core**, **spark-sql**, **spark-mllib** e connettore **spark-sql-kafka-0-10**.

3.2.1 Modello di esecuzione

Spark organizza il lavoro attorno a un *driver* e a uno o più *executor*. Il driver costruisce un DAG di *stage*; ogni stage contiene task parallelizzabili su partizioni di dati. In **Config.scala**, **SPARK_MASTER = "local[*]"** impiega tutti i core della macchina host su un singolo nodo: adeguato alla prototipazione, non rappresentativo di un cluster YARN, Kubernetes o Databricks.

Il modello driver-executor si presta a una lettura intuitiva del flusso della pipeline. Il *driver* ospita la `SparkSession`, costruisce i piani logici e coordina l'esecuzione; gli *executor* eseguono i task sulle partizioni di dati. In modalità `local[*]`, driver e executor condividono la stessa JVM, il che semplifica il debug ma non permette di osservare il comportamento di rete e shuffle tipico di un cluster distribuito. Per il training su ~80 000 righe e per micro-batch di pochi record in streaming, questa configurazione è adeguata; il collo di bottiglia, come mostrato nel Capitolo 6, risiede più nello scheduling dei micro-batch che nella capacità di parallelizzazione locale.

3.2.2 Spark SQL, Catalyst e Tungsten

Spark SQL espone un'API dichiarativa su tabelle strutturate. L'ottimizzatore **Catalyst** [4] trasforma le operazioni attraverso una catena di fasi ben definita:

1. **Parsing**: analisi sintattica della query o del piano `DataFrame`;
2. **Analysis**: risoluzione di nomi, tipi e vincoli (catalogo);
3. **Logical optimization**: regole di rewrite (pushdown di filtri e proiezioni, pruning di colonne);
4. **Physical planning**: scelta di algoritmi (broadcast join, shuffle hash join, ecc.);
5. **Preparation**: generazione di codice eseguibile.

Nel trainer, operazioni come `groupBy("type").count()` dopo la lettura CSV beneficiano di predicate e projection pushdown senza intervento manuale. Nel predictor, il parsing `from_json` e la `transform` del modello ML si integrano nello stesso planner, evitando materializzazioni intermedie superflue quando il motore può fonderle.

Tungsten [20] completa Catalyst con esecuzione orientata alla CPU e alla memoria: gestione off-heap, formato binario columnar interno e *whole-stage code generation*, che compila catene di operatori in un unico bytecode JVM. Su micro-batch ripetitivi (filtro, parsing JSON, proiezione colonne output), il codegen riduce il overhead per record rispetto a un interprete riga per riga.

Nel predictor, la catena `from_json` → `transform` → `select` viene ottimizzata come un unico piano fisico quando possibile. Catalyst può applicare *column pruning*, evitando di materializzare colonne del payload JSON non utilizzate dal modello. Sebbene nel prototipo tutte le colonne del CSV transitino nel messaggio, in un'evoluzione con payload ridotto il pruning diventerebbe un vantaggio misurabile in termini di memoria e CPU per micro-batch.

La Figura 3.1 sintetizza il percorso dal codice applicativo all'esecuzione.

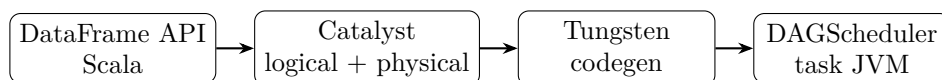


Figura 3.1: Dal codice Scala al piano eseguito: Catalyst e Tungsten nel percorso di una query Spark SQL.

3.2.3 MLlib, Pipeline API e confronto con alternative

MLlib espone algoritmi distribuiti tramite `Estimator`, `Transformer` e `Pipeline` [18]. Salvare `pipelineModel.write().save(...)` persiste in un colpo solo indicizzatore, assemblatore e Random Forest; `PipelineModel.load` garantisce che l'inferenza streaming replichi esattamente le trasformazioni del training.

Rispetto a **scikit-learn**, MLlib scala su cluster e condivide runtime con Structured Streaming, ma offre un catalogo algoritmi più ristretto e meno comodo per esperimenti rapidi su singola macchina. Rispetto a **TensorFlow/PyTorch**, MLlib non punta al deep learning end-to-end: reti neurali richiederebbero export ONNX, serving dedicato o integrazioni non native alla Pipeline API, aumentando il *technical debt* operativo descritto in letteratura per sistemi ML ibridi [25].

XGBoost4J-Spark e **GBTCClassifier** (MLlib) sono alternative plausibili per tabular data; Random Forest è stato preferito per integrazione nativa nella Pipeline, assenza di dipendenze aggiuntive e buona gestione di feature eterogenee senza scaling obbligatorio. La limitazione principale resta l'assenza di tuning sistematico (**CrossValidator**) nel prototipo, delegato a estensioni future.

Random Forest, introdotto da Breiman [6], costruisce un insieme di alberi di decisione addestrati su bootstrap del training set e su sottoinsiemi casuali di feature. La predizione finale è ottenuta per voto di maggioranza (classificazione) o media (regressione). Rispetto a un singolo albero profondo, l'ensemble riduce la varianza e migliora la generalizzazione, a patto di un costo computazionale lineare nel numero di alberi. Nel contesto Spark, il training è distribuito: ogni executor costruisce subset di alberi su partizioni locali, e il driver aggrega il modello finale. Per il dataset MIT-BIH preprocessato, 100 alberi con profondità massima 15 rappresentano un compromesso tra accuratezza e tempo di addestramento su singola macchina.

3.2.4 Structured Streaming: semantica temporale e trigger

Structured Streaming tratta il flusso come tabella illimitata aggiornata nel tempo [2]. L'API replica il batch: `readStream`, trasformazioni, `writeStream`.

Processing time ed event time. Per default, il motore ragiona in **processing time** (arrivo dei record al consumer). Con **event time** e **watermark**, Spark gestisce record in ritardo fino a una soglia configurabile; oltre quella soglia, gli eventi tardivi possono essere scartati o inviati a un side output. Nel caso ECG del simulatore, timestamp e ritmo di invio sono allineati: non è stato necessario configurare watermark, ma un Holter reale con buffer offline richiederebbe event time esplicito sul campo temporale del payload.

Trigger. I trigger principali sono: `ProcessingTime(interval)` (micro-batch periodici), `Once / AvailableNow` (elaborazione fino a fine log, utile per catch-up), e il default *as fast as possible* adottato dal predictor, che avvia un micro-batch non appena sono disponibili dati da Kafka.

Micro-batch vs continuous processing. Structured Streaming implementa, di fatto, micro-batch su Spark SQL; la modalità *continuous processing* (latenza sub-secondo) resta sperimentale e

limitata nel set di operatori. Per la tesi, il modello a micro-batch è accettabile: latenza end-to-end nell'ordine di centinaia di millisecondi resta compatibile con monitoraggio *near-real-time*, come discusso nel Capitolo 6.

Nel componente `EcgStreamingPredictor`, Structured Streaming consuma Kafka, applica `pipelineModel.transform` e scrive su Delta tramite `foreachBatch`. Questa scelta consente logica custom (`persist`, `isEmpty`, `logging`) a fronte di una gestione più esplicita del sink rispetto a `format("delta")` nativo.

Structured Streaming gestisce internamente lo stato del query plan e gli offset Kafka nel checkpoint. Dal punto di vista del programmatore, l'API rimane dichiarativa: si definisce la trasformazione e il sink, e il motore si occupa di schedulare micro-batch, gestire failure e garantire progressi. Questa astrazione riduce la complessità rispetto a consumer Kafka manuali, dove l'applicazione deve implementare esplicitamente commit degli offset, gestione dei rebalance e recovery dopo crash.

3.3 Apache Kafka: architettura publish-subscribe e disaccoppiamento

Apache Kafka organizza la messaggistica attorno a *topic*, *partizioni* e *offset* [12], [14]. I producer appendono record immutabili; i consumer leggono tramite *consumer group* con bilanciamento del carico.

3.3.1 Architettura log-centrica

Ogni topic è un log replicato e partizionato, non una coda che consuma e dimentica. Più consumer possono rileggere lo stesso flusso a offset diversi (inferenza real-time, job di riprocessamento batch, audit).

Nel progetto, `ecg-input-stream` bufferizza tra `EcgPatientSimulator` e `EcgStreamingPredictor`: il producer può inviare un battito al secondo mentre Spark elabora micro-batch con latenza variabile, senza vincolare le due velocità.

Il modello log-centrico di Kafka [12], [14] si distingue dalle code tradizionali per la persistenza duratura dei messaggi e la possibilità di rilettura. Un consumer che cade può riprendere dall'ultimo offset committato senza perdere eventi già scritti nel log; un nuovo consumer può partire da `earliest` per riprocessare l'intera storia del topic. Questa proprietà è fondamentale in pipeline analitiche dove lo stesso flusso può alimentare inferenza real-time, job di riaddestramento e audit batch, senza duplicare l'infrastruttura di ingestion.

3.3.2 Partizioni, consumer group e parallelismo

Una **partizione** è l'unità di ordinamento e parallelismo: i record con la stessa chiave finiscono nella stessa partizione, preservando l'ordine per chiave. Il numero massimo di consumer attivi in un gruppo che leggono in parallelo un topic è, in prima approssimazione, il numero di partizioni del topic. Con una sola partizione (default in sviluppo), il throughput di consumo Spark resta

limitato a un task per micro-batch; aumentare le partizioni e `spark.default.parallelism` è il primo passo verso scaling orizzontale.

Structured Streaming gestisce internamente gli offset Kafka nel checkpoint: al restart, riprende dall'ultimo offset committato insieme al batch Delta, non dal solo puntatore del consumer Java.

La scelta della chiave di partizionamento influenza l'ordinamento e il parallelismo. Nel simulatore, la chiave `beat-{index}` è arbitraria e non garantisce co-locazione per paziente o sessione. In produzione, si userebbe tipicamente l'identificativo del paziente o del dispositivo come chiave, così tutti i battiti di uno stesso soggetto finiscono nella stessa partizione e mantengono l'ordine temporale. Questo è rilevante per analisi che richiedono sequenze ordinate, come il rilevamento di raffiche ectopiche consecutive.

3.3.3 Modalità KRaft e deprecazione di ZooKeeper

Storicamente, Kafka usava ZooKeeper per metadati di cluster. KRaft (Kafka Raft) introduce un quorum interno [1], riducendo componenti da operare.

Il `docker-compose.yml` del repository configura un nodo combinato broker+controller, `quorum 1@localhost:9093`, `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1`. In produzione, broker e controller andrebbero separati con replication factor ≥ 3 .

Tabella 3.1: Confronto tra Kafka con ZooKeeper e Kafka in modalità KRaft.

Aspetto	ZooKeeper	KRaft
Componenti esterni	Sì (cluster ZK separato)	No
Tempo di failover	Secondi-decine di s	Sub-secondo (target)
Complessità operativa	Alta	Ridotta
Scalabilità metadata	Limitata	Migliorata

3.3.4 Garanzie di consegna ed exactly-once

Kafka supporta:

- **at-most-once**: messaggi possono perdersi, nessun duplicato;
- **at-least-once**: nessuna perdita, possibili duplicati al retry;
- **exactly-once** (EOS): producer idempotente + transazioni quando producer e consumer partecipano allo stesso cluster transazionale.

Spark Structured Streaming, con checkpoint e sink idempotente, punta a **exactly-once a livello di micro-batch**: ogni batch è processato una volta anche dopo restart del driver, a patto che la scrittura su Delta e il commit degli offset avvengano nella stessa transazione logica del batch. Duplicati a livello di *business key* (**record**) restano possibili se si riesegue l'intero esperimento senza deduplicazione: argomento ripreso nel Capitolo 4.

Per il prototipo, la garanzia **at-least-once** di Kafka combinata con checkpoint Spark e append Delta è sufficiente a garantire che nessun messaggio venga perso dopo un restart, accettando la possibilità di duplicati a livello di business key se l'esperimento viene ripetuto senza pulizia del checkpoint. L'exactly-once transazionale nativo di Kafka (EOS) richiederebbe producer idempotenti e consumer transazionali, con overhead aggiuntivo non giustificato in un ambiente di laboratorio single-node.

3.4 Delta Lake: il paradigma Data Lakehouse

Delta Lake estende Parquet con un *transaction log* che garantisce proprietà ACID su object store e filesystem [3]. Il paradigma *Lakehouse* unifica flessibilità del data lake e affidabilità del warehouse.

3.4.1 Limiti di Parquet puro

Parquet da solo non gestisce scritture concorrenti né versioning: due writer che appendono possono corrompere la directory. Per predizioni cliniche, dove ogni battito classificato deve essere persistito in modo auditabile, queste lacune sono critiche.

3.4.2 Transaction log, ACID e schema

Delta mantiene `_delta_log` con operazioni append, overwrite, merge, delete. Le scritture sono atomiche; l'*optimistic concurrency control* risolve conflitti tra writer. Per default vale **schema enforcement**: un append con colonne incompatibili fallisce. Con `mergeSchema=true` (schema *evolution*), nuove colonne possono essere aggiunte; utile se il payload JSON guadagna attributi, ma richiede allineamento anche sul parser `from_json` a monte.

3.4.3 Time travel, VACUUM e audit clinico

Time travel permette letture per versione (`VERSION AS OF`) o timestamp (`TIMESTAMP AS OF`), ricostruendo lo stato della tabella a un istante passato. In contesti regolati, ciò supporta audit: quale versione del log predittivo era visibile quando è stato generato un alert.

VACUUM rimuove file dati non referenziati dal log oltre una retention configurabile (default 7 giorni), bilanciando spazio disco e possibilità di rollback. Policy troppo aggressive possono impedire time travel su versioni antiche: scelta da calibrare con requisiti di conservazione clinica.

3.4.4 Integrazione con Spark

Il progetto registra le estensioni Delta nella `SparkSession`:

Listing 3.1: Configurazione Delta Lake in `SparkSession`.

```

1 .config("spark.sql.extensions",
2   "io.delta.sql.DeltaSparkSessionExtension")
3 .config("spark.sql.catalog.spark_catalog",
4   "org.apache.spark.sql.delta.catalog.DeltaCatalog")

```

La stessa configurazione compare in `EcgModelTrainer` e `EcgStreamingPredictor`; solo il predictor scrive su `spark-warehouse/ecg_predictions`. Il checkpoint Spark risiede in `spark-warehouse/checkpoints/ecg_predictions`: separare metadati di offset da dati di business è buona pratica operativa.

Il paradigma *Lakehouse* [3] unifica i vantaggi del data lake (flessibilità, costi di storage ridotti su object store) e del data warehouse (ACID, schema enforcement, query SQL performanti). Per dati clinici, dove audit e immutabilità sono requisiti ricorrenti, Delta Lake offre un compromesso attraente rispetto a Parquet puro o a database relazionali tradizionali per carichi di scrittura ad alta frequenza. Le predizioni ECG, appendate transazionalmente a ogni micro-batch, formano un log interrogabile con Spark SQL o strumenti BI, senza dover esportare verso un warehouse separato.

3.5 Sintesi: requisiti e tecnologie adottate

La Tabella 3.2 collega i requisiti dell'introduzione alle scelte tecnologiche.

Tabella 3.2: Mappatura requisiti–tecnologie.

Requisito	Tecnologia	Meccanismo
Disaccoppiamento ingestione/processing	Kafka	Topic <code>ecg-input-stream</code> , partizioni, offset
Training su dati storici	Spark MLlib	<code>EcgModelTrainer</code> , <code>split 80/20</code> , <code>Pipeline.fit</code>
Inferenza near-real-time	Structured Streaming	Micro-batch da Kafka, <code>pipelineModel.transform</code>
Persistenza affidabile	Delta Lake	Append transazionale, <code>time travel</code> , <code>foreachBatch</code>
Fault tolerance	Checkpoint Spark	<code>STREAMING_CHECKPOINT_PATH</code>
Linguaggio unificato	Scala 2.13	Batch e streaming nello stesso codebase

Nel capitolo seguente queste tecnologie sono organizzate in moduli e flussi dati concreti, orchestrati da `AppRunner`.

Capitolo 4

Progettazione e Architettura del Sistema

4.1 Pattern architetturali: Lambda, Kappa e posizionamento del progetto

Le architetture per l'elaborazione di dati in tempo reale si sono consolidate attorno a due modelli di riferimento [12], [13], [17].

L'**architettura Lambda** prevede due pipeline parallele: un *batch layer* che calcola viste corrette su dati storici e uno *speed layer* che gestisce gli aggiornamenti recenti con latenza ridotta; un *serving layer* unifica i risultati per le query. La forza della Lambda è la correttezza batch su volumi elevati; il costo è la duplicazione della logica (stesse aggregazioni implementate due volte) e la complessità operativa.

L'**architettura Kappa** propone un unico flusso di eventi: lo speed layer consuma il log; il riprocessamento storico si ottiene rileggendo il topic dall'inizio con job batch-equivalenti. Si elimina la duplicazione logica, ma servono formati immutabili, retention adeguata e capacità di ricalcolo su scala. La Figura 4.1 e la Tabella 4.1 riassumono la Lambda e il confronto con l'impostazione adottata.

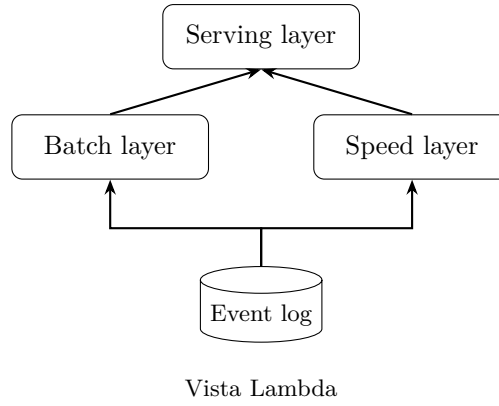


Figura 4.1: Schema concettuale dell'architettura Lambda: batch e speed layer alimentano il serving.

Il progetto di questa tesi adotta un **ibrido pragmatico**. L'addestramento avviene in batch off-line (**EcgModelTrainer**); l'inferenza opera in streaming (**EcgStreamingPredictor**). Non esiste uno speed layer che ricalcoli aggregati su finestra mobile: il modello è un artefatto statico ricaricato dal filesystem fino a un nuovo training. La configurazione è vicina alla Lambda con confine netto training/serving, pratica comune nei sistemi ML operativi dove la frequenza di retraining è ordini di grandezza inferiore a quella di inferenza [25].

Tabella 4.1: Confronto sintetico Lambda, Kappa e impostazione del progetto.

Aspetto	Lambda	Kappa	Questo progetto
Logica di calcolo	Duplicata (batch + speed)	Unificata sul log	Train vs infer separati
Latenza inferenza	Bassa (speed layer)	Bassa	Bassa (streaming)
Correttezza storica	Alta (batch)	Ricalcolo dal log	Training batch su CSV
Complessità ops	Alta	Media	Bassa (prototipo)
Model serving	Non specificato	Non specificato	Artefatto ML su FS

Rispetto alla Kappa pura, manca la possibilità di riaddestrare rileggendo soltanto Kafka: il training legge CSV storici, non il topic. Rispetto alla Lambda pura, manca un serving layer che fondo viste batch e real-time; le predizioni sono scritte direttamente su Delta Lake dal predictor.

La letteratura sui sistemi ML in produzione [25] evidenzia come la separazione tra training e serving sia una delle fonti principali di *technical debt*: pipeline di feature engineering diverse, versioni di librerie non allineate, assenza di monitoraggio del drift. Il progetto mitiga parte di questo rischio usando la stessa **PipelineModel** serializzata per batch e streaming, e lo stesso schema JSON derivato dal CSV di training. Restano tuttavia gap noti: nessun registry versionato del modello, nessun monitoraggio automatico della distribuzione delle feature in ingresso, nessun meccanismo di rollback se un nuovo modello degrada le metriche.

4.2 Panoramica della pipeline

La pipeline si articola in tre moduli funzionali e un orchestratore:

1. **Batch layer** — `EcgModelTrainer`: legge CSV storici, addestra la pipeline ML, salva il modello su filesystem.
2. **Ingestion layer** — `EcgPatientSimulator`: simula un dispositivo edge, pubblica battiti JSON su Kafka.
3. **Streaming layer** — `EcgStreamingPredictor`: consuma Kafka, classifica, persiste su Delta Lake.
4. **Orchestratore** — `AppRunner`: coordina l'avvio sequenziale dei componenti.

La Figura 4.2 mostra il flusso logico dei dati; la Figura 4.3 ne dettaglia la sequenza temporale di avvio.

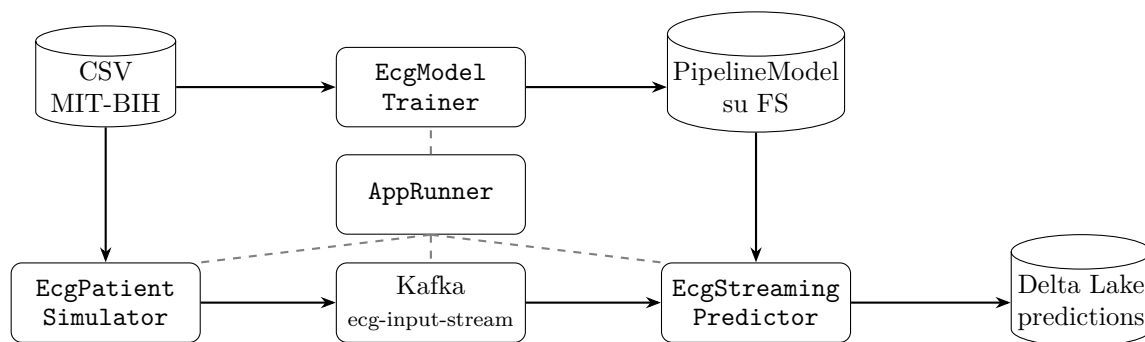


Figura 4.2: Flusso logico: training batch, ingestion simulata e inferenza streaming. Linee tratteggiate: orchestrazione `AppRunner`.

4.2.1 Flusso granulare dei dati

In sintesi: il trainer costruisce il modello da CSV; il simulatore pubblica JSON su Kafka; il predictor deserializza, classifica e appende su Delta con checkpoint. Il dettaglio implementativo (listing, `foreachBatch`, configurazione) è nel Capitolo 5. Il modello ML (`PipelineModel.load`) è l'unico artefatto condiviso tra batch e streaming.

Formato del messaggio Kafka. Ogni record pubblicato dal simulatore è una stringa JSON che serializza una riga del CSV: identificativo `record`, etichetta `type` (ground truth disponibile per valutazione, ma non usata in inferenza), e le 32 feature numeriche con prefissi `0_` e `1_`. Il consumer Spark non riceve metadati Kafka aggiuntivi oltre a `key`, `value`, `timestamp` e `partition`; la logica applicativa opera esclusivamente sul campo `value` parsato. In un deployment reale, si potrebbe arricchire il payload con timestamp di acquisizione sul dispositivo, identificativo paziente pseudonimizzato e versione del firmware di estrazione feature, campi assenti nel prototipo ma utili per tracciabilità e debug.

Persistenza delle predizioni. Ogni micro-batch processato produce righe Delta con colonne `record`, `true_label`, `prediction` e `probability`. La colonna `true_label` deriva dal campo `type` del messaggio JSON e consente valutazione ex post confrontando predizione e etichetta reale, senza richiedere un join con il dataset di training. La colonna `probability` contiene il vettore di probabilità per classe, utile per analisi di calibrazione o per soglie operative diverse dall'argmax implicito del classificatore.

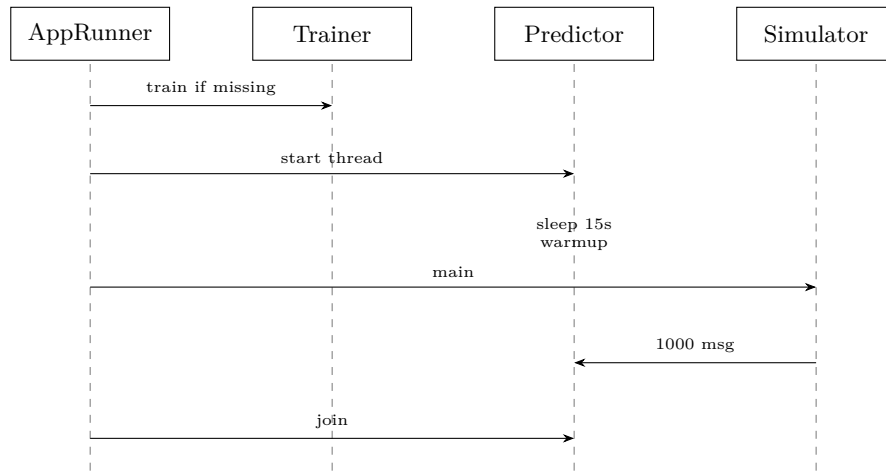


Figura 4.3: Sequenza di avvio orchestrata da **AppRunner**: training condizionale, predictor in thread demone, attesa, simulatore, join.

4.3 Vista fisica e deployment

In ambiente sperimentale, l'intera pipeline gira su un singolo host di sviluppo. La Figura 4.4 illustra la disposizione fisica.

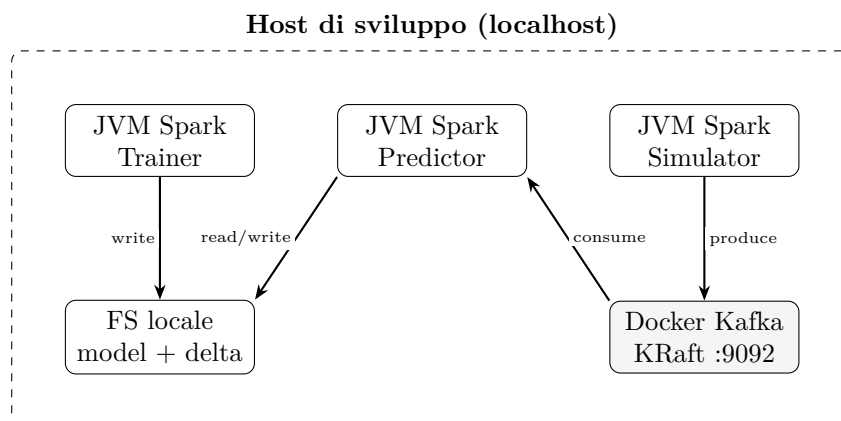


Figura 4.4: Deployment mono-nodo: tre processi JVM Spark (`local[*]`) e un container Docker per Kafka.

4.3.1 Limiti del setup mono-nodo

`SPARK_MASTER = "local[*]"` impiega tutti i core su un'unica macchina; Kafka ha replication factor 1. Un crash del host interrompe l'intera pipeline. In produzione: cluster Spark dedicato, Kafka multi-broker con $RF \geq 3$, Delta su object store (S3, ADLS). Il setup attuale è deliberatamente minimalista per dimostrare l'integrazione end-to-end.

Un deployment distribuito tipico separerebbe i ruoli in modo più netto: il trainer girerebbe su un cluster Spark dedicato o su un servizio schedato (Databricks, EMR); il predictor su un cluster streaming con executor sempre attivi; Kafka su un cluster multi-broker con replication factor ≥ 3 ; Delta su object store (S3, Azure Data Lake Storage) con accesso da più job. Il filesystem locale usato nel prototipo per modello e predizioni è un sostituto didattico dello storage condiviso, ma non offre durabilità né condivisione tra nodi. La Figura 4.4 va letta come vista *as-is* del laboratorio, non come target di produzione.

4.4 Fase di addestramento: `EcgModelTrainer`

Il modulo batch implementa il ciclo completo di machine learning supervisionato.

4.4.1 Flusso operativo

1. Lettura CSV da `DATASET_PATH` con header e inferenza schema.
2. Pulizia: `na.drop()` elimina righe con valori mancanti.
3. Split train/test 80/20 con `seed = 42L` per riproducibilità.
4. Costruzione pipeline: `StringIndexer` $\rightarrow$ `VectorAssembler` $\rightarrow$ `RandomForestClassifier`.
5. Addestramento (`pipeline.fit`), valutazione su test set, salvataggio su `ML_MODEL_PATH`.

4.4.2 Scelta del classificatore e alternative

Random Forest è configurato con `numTrees=100` e `maxDepth=15` [6]. La scelta risponde alla natura tabellare delle feature (intervalli `pre-RR/post-RR`, morfologia QRS e intervalli PQ/QT/ST su due derivazioni) e alla capacità degli ensemble di catturare interazioni non lineari senza normalizzazione rigorosa.

Alternative valutabili nello stesso stack Spark includono:

- **GBTClassifier** (gradient boosting nativo MLlib): spesso competitivo su dati tabellari, con tempi di training superiori e maggiore rischio di overfitting senza tuning;
- **LinearSVC / LogisticRegression**: baseline interpretabili, meno adatte se le interazioni tra feature RR e morfologia QRS sono forti;
- **Reti neurali**: richiederebbero export e serving fuori dalla Pipeline MLlib, aumentando la complessità operativa senza beneficio dimostrato su questo dataset nel prototipo.

Feature importance. Random Forest fornisce, per ogni albero, una misura di impurezza ridotta per split; aggregando sui 100 alberi si ottiene un ranking delle feature utile per dialogare con il dominio clinico (quali descrittori RR o QRS guidano le confusions tra N e V). Non è implementata nel codice attuale, ma resta un'estensione naturale per migliorare l'interpretabilità senza cambiare architettura.

Cross-validation. MLlib espone `CrossValidator` e `TrainValidationSplit` per grid search su `numTrees` e `maxDepth`. Non sono stati usati: il focus della tesi è l'integrazione streaming, e il dataset preprocessato ha dimensione moderata; un tuning sistematico migliorerebbe la generalizzazione a costo di tempi di sperimentazione molto più lunghi.

Il flusso di training può essere eseguito indipendentemente dal resto della pipeline. Se il modello esiste già in `ML_MODEL_PATH`, `AppRunner` salta la fase di addestramento e procede direttamente all'inferenza. Questa logica condizionale consente di iterare rapidamente sul predictor e sul simulatore senza ripetere il fit del Random Forest, riducendo i tempi di sviluppo. In produzione, il training sarebbe schedato (notturno, settimanale) e il predictor caricherebbe sempre l'ultima versione validata dal registry, non un path statico sul filesystem locale.

4.4.3 Output del trainer

Il modello salvato include metadata, stages serializzati e statistiche dell'indicizzatore. In streaming, etichette mai viste in training sono gestite da `setHandleInvalid("keep")`, che assegna un indice dedicato anziché sollevare eccezioni: robustezza operativa a scapito di predizioni potenzialmente poco informative su classi *unknown*.

4.5 Simulazione IoT: EcgPatientSimulator

Il simulatore riproduce un dispositivo wearable che trasmette battiti verso un broker centralizzato.

4.5.1 Logica di produzione

1. Lettura di al massimo 1000 record dal CSV (`.limit(1000)`).
2. Serializzazione JSON tramite `to_json(struct(col("*")))`.
3. Raccolta in memoria (`.collect()`) e invio sequenziale con `KafkaProducer`.
4. Intervallo di un secondo tra messaggi (`Thread.sleep(1000)`).

L'uso di `KafkaProducer` Java puro, anziché `df.writeStream.format("kafka")`, avvicina il simulatore a un producer esterno a Spark, come avverrebbe su firmware edge senza motore distribuito.

4.5.2 Schema del payload e impatto sulla banda

Ogni messaggio serializza *tutte* le colonne del CSV: identificativo **record**, etichetta **type**, feature numeriche e metadati. La chiave **beat-{index}** serve al debug ma non è sfruttata per partizionamento semantico (paziente, sessione). In produzione, si potrebbe ridurre il payload inviando solo le feature richieste dal modello, con risparmio di banda e minor superficie di esposizione dati; il contratto andrebbe però versionato esplicitamente (Schema Registry, vedi §4.6.2).

4.5.3 Estensioni e limiti

Il rate fisso di 1 msg/s è una semplificazione rispetto a un Holter reale (jitter, burst dopo riconnessione, gap di trasmissione). Estensioni naturali: intervallo casuale gaussiano intorno alla frequenza cardiaca; simulazione di disconnessioni; ripresa con **startingOffsets** e idempotenza lato consumer.

`.collect()` materializza 1000 record sul driver: accettabile in demo, inaccettabile per flussi continui. Questi limiti sono consapevoli e discussi nelle conclusioni.

Il simulatore legge dal CSV le prime 1000 righe nell'ordine in cui compaiono nel file, senza mescolamento casuale né campionamento stratificato per classe. Ciò implica che la sequenza di messaggi Kafka riflette la distribuzione locale del file, non necessariamente quella globale del dataset. Per test di stress o per valutare il comportamento su classi rare, si potrebbe filtrare il CSV prima della pubblicazione o aumentare il limite, accettando tempi di esecuzione più lunghi proporzionali al rate di 1 msg/s.

4.6 Inferenza in tempo reale: EcgStreamingPredictor

Il predictor è il cuore operativo della pipeline in fase di serving.

4.6.1 Bootstrap dello schema JSON

Aniché hardcodare uno **StructType**, il codice legge lo schema dal CSV di training:

Listing 4.1: Inferenza schema da CSV per `from_json`.

```
1 val jsonSchema = spark.read
2   .option("header", "true")
3   .option("inferSchema", "true")
4   .csv(datasetPath)
5   .schema
```

Vantaggio: nessuna duplicazione manuale tra producer e consumer. Svantaggio: accoppiamento implicito al layout del CSV; una modifica delle colonne richiede riavvio coerente di simulatore e predictor.

4.6.2 Schema Registry e contratto dati

In ambienti industriali, **Confluent Schema Registry** (o equivalenti) gestisce schemi Avro, JSON Schema o Protobuf con regole di compatibilità (*backward, forward*). Il producer registra la versione *v* del messaggio; il consumer la risolve a runtime, fallendo in modo controllato se il contratto non è compatibile. Per il prototipo accademico, l'inferenza da CSV è sufficiente; per produzione clinica, un contratto esplicito riduce il rischio di parsing silenzioso con campi mancanti o tipi errati dopo aggiornamento firmware del dispositivo.

4.6.3 Pipeline di inferenza

1. `readStream.format("kafka")` con `startingOffsets=earliest` (solo al primo avvio senza checkpoint).
2. Cast del payload a stringa, parsing con `from_json`.
3. Trasformazione tramite `pipelineModel.transform`.
4. Proiezione su `record`, `true_label`, `prediction`, `probability`.
5. Scrittura su Delta Lake via `foreachBatch` con checkpoint.

4.6.4 Ciclo di vita del modello in produzione

Il pattern attuale — modello su directory locale, caricato all'avvio del predictor — è tipico dei prototipi ma insufficiente per operazioni mature [25]:

- **Model serving**: registry (MLflow, servizio REST) con versioni immutabili e metadati di training;
- **A/B testing**: routing di una frazione del traffico su modello candidato;
- **Model drift**: monitoraggio distribuzione feature e calo di metriche, che innesca retraining;
- **Hot-swap**: sostituzione del modello senza fermare il job streaming (richiede doppio caricamento o query restart controllato).

Nessuno di questi meccanismi è implementato; il trainer sovrascrive `ML_MODEL_PATH` e il predictor deve essere riavviato per caricare una nuova versione.

4.6.5 Fault tolerance e exactly-once

Il checkpoint in `STREAMING_CHECKPOINT_PATH` registra offset Kafka e metadati del query plan. In caso di failure del driver, il restart riprende dall'ultimo checkpoint committato. Checkpoint + append idempotente su Delta garantisce exactly-once a livello di micro-batch; duplicati di business key restano possibili se si ripete l'esperimento senza deduplicazione.

4.7 Orchestrazione: AppRunner

AppRunner coordina l'avvio in sequenza logica:

1. Verifica esistenza del modello in `ML_MODEL_PATH`; se assente, invoca `EcgModelTrainer.main`.
2. Avvia `EcgStreamingPredictor` in un thread demone (`setDaemon(true)`).
3. Attende 15 secondi (`Thread.sleep(15000)`) per il warmup di Spark Streaming.
4. Avvia `EcgPatientSimulator`.
5. Attende la terminazione del thread predictor (`predictorThread.join()`).

L'attesa di 15 secondi è un'euristica che evita che i primi messaggi arrivino prima che la query streaming sia attiva; in produzione andrebbe sostituita da health-check (`isActive`, connettività Kafka, readiness del consumer group).

AppRunner rappresenta il punto di ingresso unico dell'applicazione (`sbt run`). La sequenza di avvio riflette dipendenze logiche: il modello deve esistere prima che il predictor possa caricarlo; il predictor deve essere attivo prima che il simulatore inizi a pubblicare, altrimenti i primi messaggi potrebbero accumularsi nel topic senza essere consumati immediatamente. Il thread demone del predictor garantisce che la JVM non termini quando il main thread completa il simulatore, ma resta in attesa di `awaitAnyTermination` fino a interruzione esplicita o fine dello stream. In assenza di `join()`, il programma chiuderebbe la JVM mentre la query streaming è ancora in esecuzione, perdendo predizioni non ancora scritte su Delta.

4.8 Requisiti non funzionali

4.8.1 Disaccoppiamento

Kafka isola producer e consumer: il simulatore può terminare dopo 1000 messaggi mentre il predictor consuma il backlog. Il trainer batch è indipendente dal flusso streaming, eseguibile on-demand o schedulato (cron, Airflow).

Il disaccoppiamento temporale è particolarmente utile in scenari di manutenzione: è possibile fermare il predictor per aggiornare il modello o modificare la configurazione Spark senza interrompere l'acquisizione sul dispositivo edge, che continua a pubblicare sul topic fino al raggiungimento della retention configurata. Analogamente, un picco di carico sul predictor non si propaga al producer, che opera a velocità indipendente.

4.8.2 Idempotenza e consistenza

Delta Lake append con checkpoint Spark produce un log immutabile di predizioni. Ripetere l'esperimento con gli stessi dati genera righe duplicate (non idempotenza su `record`); un `MERGE`

Delta su chiave risolverebbe il problema. In termini concreti, l'idempotenza della pipeline dipende dal checkpoint di Structured Streaming: se il predictor si arresta e viene riavviato, Spark riprende dall'ultimo offset committato, evitando la rielaborazione dei micro-batch già scritti su Delta. Il log append-only di Delta Lake consente comunque di rilevare eventuali duplicati a posteriori tramite query SQL sulla colonna `record`, anche senza ricorrere a `MERGE`. In ambito clinico, la tracciabilità completa di ogni predizione — incluse eventuali ripetizioni — può risultare preferibile rispetto a una deduplicazione silenziosa, poiché garantisce che nessun risultato venga scartato senza possibilità di audit.

4.8.3 Scalabilità

La scalabilità orizzontale passa dal parallelismo delle partizioni Kafka e dal numero di executor Spark. Con topic a partizione singola, il consumo resta serializzato; aumentare partizioni e parallelismo sblocca throughput superiore, come stimato nel Capitolo 6. Ogni partizione Kafka viene assegnata ad al più un task Spark per micro-batch: il grado di parallelismo effettivo è quindi limitato dal minimo tra il numero di partizioni e il numero di core disponibili negli executor. La configurazione a partizione singola adottata nel prototipo serializza il consumo, ma preserva l'ordine temporale dei messaggi all'interno della partizione, proprietà rilevante quando la sequenza dei campioni ECG ha significato clinico. Con un partizionamento basato sull'identificativo del paziente, flussi provenienti da dispositivi diversi potrebbero essere elaborati in parallelo, mantenendo al contempo l'ordinamento per-paziente necessario alla corretta ricostruzione del segnale.

4.8.4 Evolvibilità dello schema

L'approccio JSON implicito è fragile: Delta può evolvere lo schema in append, ma `from_json` fallisce su tipi incompatibili o campi obbligatori mancanti. Formati schema-first (Avro, Protobuf) e Registry mitigano il rischio. Se, ad esempio, una nuova versione del firmware aggiungesse una colonna al payload JSON, `from_json` con lo schema precedente scarterebbe silenziosamente il campo aggiuntivo; viceversa, la rimozione di un campo produrrebbe valori null nella colonna corrispondente. Il rischio principale è che il modello si attende 32 feature numeriche: un cambiamento dello schema potrebbe introdurre null che degradano le predizioni senza generare errori espliciti, poiché `setHandleInvalid("keep")` nel `VectorAssembler` gestisce i valori mancanti in modo silente. Un formato schema-first intercetterebbe la discrepanza già in fase di deserializzazione, rendendo il problema visibile prima che raggiunga il modello.

4.8.5 Observability

Il prototipo si affida a log su console (`println`, `show` nei batch) e alla Spark UI locale. In produzione servirebbero metriche esportate (throughput consumer, lag Kafka, latenza micro-batch, error rate parsing) verso Prometheus/Grafana o equivalenti cloud, con tracing distribuito per correlare `batchId` e offset. Senza observability strutturata, diagnosticare colli di bottiglia o drift operativo resta difficile oltre la demo controllata. A titolo di esempio, il monitoraggio del consumer lag di Kafka — la differenza tra l'offset più recente nel topic e l'offset dell'ultimo messaggio consumato — consentirebbe di verificare in tempo reale se il predictor mantiene il passo con l'ingestione; un lag crescente segnalerebbe backpressure o un collo di bottiglia nella

fase di inferenza. Spark espone l'interfaccia `StreamingQueryListener`, che notifica per ogni micro-batch metriche quali tempo di elaborazione, numero di righe in ingresso e aggiornamenti di stato; tali dati potrebbero essere inoltrati a un sistema di monitoraggio esterno senza alterare la logica applicativa della pipeline.

4.8.6 Sicurezza

L'istanza Kafka in Docker espone `PLAINTEXT` su localhost: nessuna autenticazione SASL, nessun TLS. I dati ECG, anche come feature, sono informazioni sanitarie sensibili: un deployment reale richiederebbe cifratura in transito (TLS tra edge, broker e Spark), controllo accessi al topic, cifratura at-rest su Delta e policy di retention allineate al GDPR. Il prototipo accademico omette questi controlli per ridurre la frizione sperimentale; vanno elencati esplicitamente come gap verso un sistema clinicamente deployabile. Nel prototipo, tutto il traffico transita esclusivamente sulla rete Docker locale (localhost), circostanza che limita la superficie di attacco alla macchina host; in un deployment multi-host, il traffico Kafka non cifrato esporrebbe le feature ECG sulla rete, rendendole intercettabili. I file Delta Lake su filesystem locale ereditano i controlli di accesso del sistema operativo, ma non dispongono di autorizzazione a grana fine — ad esempio a livello di riga o di colonna — che un sistema clinico richiederebbe per garantire che solo il personale autorizzato possa accedere ai dati di specifici pazienti o a determinate classi di predizioni.

4.8.7 Manutenibilità e modularità

La separazione in package `batch`, `streaming` e `utils` consente di modificare un componente senza ricompilare l'intero progetto, purché i contratti (`Config`, schema JSON, path del modello) restino stabili. Ogni modulo espone un `main` indipendente, eseguibile separatamente per debug: il trainer senza simulatore, il predictor con messaggi già presenti nel topic, il simulatore senza consumer attivo. Questa modularità facilita lo sviluppo incrementale e la diagnosi di failure isolati, riducendo il tempo necessario per identificare quale stadio della pipeline introduce un errore.

Il Capitolo 5 approfondisce configurazione e codice; il Capitolo 6 raccoglie le misure sperimentali.

Capitolo 5

Implementazione e Dettagli Tecnici

5.1 Organizzazione del codice sorgente

Il repository `ecg-streaming-classifier` segue una struttura piatta, coerente con la dimensione del progetto:

```
src/main/scala/  
  AppRunner.scala  
  batch/EcgModelTrainer.scala  
  streaming/EcgPatientSimulator.scala  
  streaming/EcgStreamingPredictor.scala  
  utils/Config.scala
```

Il package root è `it.utiu.thesis`, definito in `build.sbt` tramite `idePackagePrefix`. I sottopackage `batch`, `streaming` e `utils` separano responsabilità senza introdurre layer di astrazione superflui.

5.1.1 Centralizzazione della configurazione

Tutti i parametri operativi risiedono in `utils.Config`:

Listing 5.1: Parametri centralizzati in `Config.scala`.

```
1 object Config {  
2   val KAFKA_BOOTSTRAP_SERVERS = "localhost:9092"  
3   val KAFKA_TOPIC = "ecg-input-stream"  
4   val SPARK_MASTER = "local[*]"  
5  
6   val DATASET_PATH = "/home/andrea/data/ecg/*.csv"  
7   val ML_MODEL_PATH = "ml-model/ecg-pipeline-model"  
8  
9   val DELTA_PREDICTIONS_PATH = "spark-warehouse/ecg_predictions"  
10  val STREAMING_CHECKPOINT_PATH =  
11    "spark-warehouse/checkpoints/ecg_predictions"  
12 }
```

La configurazione hardcoded è un limite riconosciuto del prototipo. In un'evoluzione produttiva, i path e gli endpoint andrebbero esternalizzati tramite Typesafe Config (HOCON), variabili d'ambiente o un servizio di configurazione centralizzato. Il path del dataset punta a un percorso assoluto Linux (`/home/andrea/data/ecg/*.csv`): su Windows o macOS va adattato manualmente prima dell'esecuzione.

La struttura piatta del repository riflette la dimensione del progetto: quattro moduli funzionali e un file di configurazione, senza layer di astrazione intermedie (repository pattern, service layer, dependency injection). Per un prototipo accademico questa scelta è appropriata: ogni file corrisponde a un componente architetturale identificabile nel Capitolo 4, e il lettore può navigare dal diagramma logico al codice senza attraversare gerarchie di package profonde. Un'evoluzione verso produzione potrebbe introdurre moduli SBT separati (`trainer`, `predictor`, `simulator`) con artefatti JAR indipendenti, deployabili su cluster distinti.

5.2 Build e gestione delle dipendenze

Il progetto è gestito con **SBT 1.12.4** e Scala **2.13.18**. Il file `build.sbt` fissa le versioni dello stack analitico:

Listing 5.2: Dipendenze principali in `build.sbt`.

```
1 val sparkVersion = "4.1.1"
2 val deltaVersion = "4.1.0"
3
4 libraryDependencies += Seq(
5   "org.apache.spark" %% "spark-core" % sparkVersion,
6   "org.apache.spark" %% "spark-sql" % sparkVersion,
7   "org.apache.spark" %% "spark-mllib" % sparkVersion,
8   "org.apache.spark" %% "spark-sql-kafka-0-10" % sparkVersion,
9   "io.delta" %% "delta-spark" % deltaVersion,
10 )
```

Spark e Delta condividono la stessa release train (4.1.x): questo riduce incompatibilità tra catalogo Delta e runtime SQL. Il connettore `spark-sql-kafka-0-10` è allineato alla versione Spark, requisito per Structured Streaming su Kafka. Non sono presenti dipendenze di test (`scalatest`) né plugin di packaging (`sbt-assembly`): coerente con un prototipo didattico, ma limitante per CI automatizzata.

`idePackagePrefix := Some("it.utiu.thesis")` imposta il package root per l'IDE senza alterare la struttura delle directory sotto `src/main/scala/`.

L'assenza di `sbt-assembly` implica che l'esecuzione avviene tramite `sbt run`, che risolve le dipendenze Spark e Delta dal classpath SBT. In un deployment cluster, sarebbe necessario costruire un *fat JAR* con tutte le dipendenze (o usare Spark submit con `-packages`), e gestire i conflitti di versione tra connettori Kafka, Delta e runtime Spark. Per il prototipo locale, `sbt run` è sufficiente e riduce la complessità di build; la transizione verso packaging production-grade è una estensione naturale non affrontata in questa tesi.

5.3 Infrastruttura Docker e broker Kafka

Kafka è avviato tramite `docker-compose.yml` con immagine `apache/kafka:latest` in modalità **KRaft** su un singolo nodo (`broker,controller`). I parametri salienti sono: `KAFKA_CONTROLLER_QUORUM_VOTERS=1@localhost:9093`, `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=1`, listener `PLAINTEXT` su porta 9092.

In laboratorio questo basta a esercitare producer e consumer locali. In produzione servirebbero almeno tre broker, replication factor ≥ 3 e separazione dei ruoli controller/broker; altrimenti un guasto al container interrompe l'intera ingestione.

Spark non è containerizzato: gira sulla JVM host con `local[*]`. La scelta semplifica il debug (log unificati, Spark UI su 4040) a scapito del disaccoppiamento infrastrutturale tipico dei deployment cloud-native.

Il file `docker-compose.yml` avvia un singolo container Kafka con ruoli broker e controller combinati. I listener sono configurati per `PLAINTEXT` su `localhost:9092`, senza autenticazione SASL né TLS. Per avviare l'infrastruttura è sufficiente `docker-compose up -d` dalla root del repository; lo stop con `docker-compose down` rimuove il container ma non i volumi, preservando i topic e gli offset tra sessioni di sviluppo. In caso di corruzione dello stato Kafka o di necessità di reset completo, è possibile ricreare il container con `docker-compose down -v` e ripartire da offset `earliest`, previa cancellazione del checkpoint Spark per evitare incongruenze tra offset committati e stato del broker.

5.4 Configurazione SparkSession

Trainer, predictor e simulatore creano ciascuno una propria `SparkSession`, ripetendo la stessa configurazione Delta:

Listing 5.3: Configurazione comune della sessione Spark.

```
1 SparkSession.builder()
2   .appName("EcgModelTrainer") // o EcgStreamingPredictor / EcgPatientSimulator
3   .master(Config.SPARK_MASTER)
4   .config("spark.driver.host", "127.0.0.1")
5   .config("spark.driver.bindAddress", "127.0.0.1")
6   .config("spark.sql.extensions",
7     "io.delta.sql.DeltaSparkSessionExtension")
8   .config("spark.sql.catalog.spark_catalog",
9     "org.apache.spark.sql.delta.catalog.DeltaCatalog")
10  .getOrCreate()
```

`SPARK_MASTER = "local[*]"` assegna tutti i core della macchina a un unico executor logico: adeguato per ~100 000 righe CSV e micro-batch piccoli, insufficiente per carichi multi-paziente senza cluster.

Parametri di tuning non sono stati variati (`spark.sql.shuffle.partitions`, memoria driver/executor): su dataset tabellari di questa dimensione i default Spark 4.x sono generalmente sufficienti; su cluster andrebbero calibrati in funzione del numero di partizioni Kafka e della cardinalità del modello.

Trainer, predictor e simulatore creano ciascuno una propria `SparkSession` con `getOrCreate()`, che riusa la sessione esistente se già attiva nella JVM. In `AppRunner`, trainer e predictor condividono potenzialmente la stessa sessione se eseguiti nello stesso processo, mentre il simulatore ne crea una terza quando viene avviato dal main thread. Questa molteplicità di sessioni non causa conflitti nel prototipo, ma in un deployment più rigoroso si potrebbe centralizzare la creazione della sessione in un modulo condiviso, evitando configurazioni duplicate e garantendo un unico punto di modifica per parametri Spark globali.

5.5 Data pipeline e machine learning

5.5.1 Preprocessing e feature engineering

Il trainer legge CSV con header e inferenza automatica dello schema. La pulizia è minimale: `rawDF.na.drop()` elimina righe con qualsiasi valore nullo, senza imputazione.

La selezione delle colonne feature avviene per esclusione:

Listing 5.4: Selezione colonne feature in `EcgModelTrainer`.

```
1 val featureCols = cleanDF.columns.filter(colName =>
2   colName != "record" && colName != "type" && colName != "label"
3 )
```

Le colonne `record` (identificativo traccia) e `type` (etichetta testuale) vengono escluse dal vettore numerico. L'esclusione di `label` è difensiva: evita leakage se la colonna fosse presente nel CSV con valori numerici già indicizzati.

Lo `StringIndexer` converte `type` in `label` numerico:

Listing 5.5: Pipeline ML: indicizzazione, assemblaggio, classificazione.

```
1 val indexer = new StringIndexer()
2   .setInputCol("type")
3   .setOutputCol("label")
4   .setHandleInvalid("keep")
5
6 val assembler = new VectorAssembler()
7   .setInputCols(featureCols)
8   .setOutputCol("features")
9   .setHandleInvalid("keep")
10
11 val rf = new RandomForestClassifier()
12   .setLabelCol("label")
13   .setFeaturesCol("features")
14   .setNumTrees(100)
15   .setMaxDepth(15)
16
17 val pipeline = new Pipeline()
18   .setStages(Array(indexer, assembler, rf))
```

`setHandleInvalid("keep")` su entrambi gli stadi è una scelta pragmatica. In training, etichette o feature mancanti ricevono un bucket dedicato anziché causare eccezioni. In streaming,

battiti con feature incomplete possono comunque transitare nella pipeline, producendo predizioni potenzialmente inaffidabili. Un approccio più rigoroso prevedrebbe filtraggio a monte (**drop** o quarantena su topic separato) anziché tolleranza silenziosa.

Il numero di feature effettive dipende dalle colonne presenti nel CSV: con 34 colonne totali e esclusione di `record`, `type` e `label`, il vettore numerico contiene 32 attributi (16 per derivazione). Il `VectorAssembler` concatena questi valori in un unico vettore denso senza normalizzazione esplicita. Random Forest non richiede scaling delle feature, a differenza di algoritmi basati su distanze o gradienti; questa proprietà semplifica la pipeline e riduce il rischio di mismatch tra statistiche di normalizzazione calcolate in training e applicate in streaming.

5.5.2 Split, addestramento e valutazione

Listing 5.6: Split train/test e valutazione.

```
1 val Array(trainingData, testData) =
2   cleanDF.randomSplit(Array(0.8, 0.2), seed = 42L)
3
4 val pipelineModel = pipeline.fit(trainingData)
5
6 val predictions = pipelineModel.transform(testData)
7
8 val evaluator = new MulticlassClassificationEvaluator()
9   .setLabelCol("label")
10  .setPredictionCol("prediction")
11  .setMetricName("accuracy")
12
13 val accuracy = evaluator.evaluate(predictions)
```

Lo `seed = 42L` garantisce riproducibilità dello split. L'evaluator calcola solo l'accuratezza globale; metriche per-classe (precision, recall, F1) richiederebbero un `MulticlassClassificationEvaluator` per ogni metrica o l'uso di `BinaryClassificationMetrics` su coppie one-vs-rest. Il Capitolo 6 estende l'analisi con metriche più granulari.

5.5.3 Iperparametri Random Forest

`numTrees=100` e `maxDepth=15` non sono stati ottimizzati con grid search. La scelta riflette un compromesso documentato in letteratura [6]: 100 alberi offrono varianza sufficientemente bassa senza costi eccessivi; profondità 15 limita overfitting su dataset di dimensione moderata. Un tuning sistematico con `CrossValidator` o `TrainValidationSplit` migliorerebbe la generalizzazione, a costo di tempi di training più lunghi.

5.5.4 Serializzazione e persistenza del modello

Listing 5.7: Salvataggio pipeline su filesystem.

```
1 pipelineModel.write.overwrite().save(modelPath)
```

Spark ML salva la pipeline in una directory con struttura:

```
ml-model/ecg-pipeline-model/  
  metadata/  
  stages/  
    0_StringIndexer_.../  
    1_VectorAssembler_.../  
    2_RandomForestClassificationModel_.../
```

Il flag `overwrite()` consente retraining senza rimozione manuale della directory precedente. In produzione, versionare i modelli (MLflow, DVC) eviterebbe sovrascritture accidentali e permetterebbe rollback.

La directory del modello include metadata Spark ML, stages serializzati e statistiche dell'indicizzatore (`StringIndexer`). Al caricamento con `PipelineModel.load`, il predictor ottiene un oggetto immutabile pronto per `transform`, senza rieseguire fit o accesso al dataset di training. Questa separazione è il meccanismo che rende l'inferenza streaming indipendente dalla fase batch: il predictor non legge il CSV se non per inferire lo schema JSON, e non accede ai dati di training per la classificazione.

5.6 Gestione dello streaming

5.6.1 Parsing JSON e schema dinamico

Il consumer Kafka decodifica il payload in due passaggi:

Listing 5.8: Parsing messaggi Kafka in `EcgStreamingPredictor`.

```
1 val kafkaStreamDF = spark.readStream  
2   .format("kafka")  
3   .option("kafka.bootstrap.servers", Config.KAFKA_BOOTSTRAP_SERVERS)  
4   .option("subscribe", Config.KAFKA_TOPIC)  
5   .option("startingOffsets", "earliest")  
6   .load()  
7  
8 val parsedStreamDF = kafkaStreamDF  
9   .selectExpr("CAST(value AS STRING) as json_string")  
10  .select(from_json(col("json_string"), jsonSchema).alias("data"))  
11  .select("data.*")
```

`startingOffsets=earliest` garantisce che, al primo avvio, vengano consumati tutti i messaggi presenti nel topic. In restart con checkpoint, Spark riprende dall'ultimo offset committato, ignorando questa opzione.

L'alternativa professionale — schema Avro registrato su Confluent Schema Registry — offrirebbe validazione strict, compatibilità backward/forward e evoluzione controllata. Per un prototipo accademico, l'inferenza schema da CSV riduce il boilerplate a costo di accoppiamento implicito.

5.6.2 Trasformazione e proiezione output

Listing 5.9: Inferenza e selezione colonne output.

```
1 val predictionsStreamDF = pipelineModel.transform(parsedStreamDF)
2
3 val finalStreamDF = predictionsStreamDF.select(
4     col("record"),
5     col("type").alias("true_label"),
6     col("prediction"),
7     col("probability")
8 )
```

La colonna `probability` contiene un vettore sparse di probabilità per classe. In analisi successive, estrarre la probabilità della classe predetta richiede UDF o parsing del vettore.

5.6.3 Checkpointing e fault tolerance

L'Algoritmo 1 descrive la logica di `foreachBatch`.

Algorithm 1: Elaborazione micro-batch con `foreachBatch`

Input: Micro-batch B_i , identificativo $batchId$, path Delta P , checkpoint C

Output: Predizioni persistite, offset committati

```
1  $B_i$ .persist();
2 if  $B_i$  non è vuoto then
3     Logga  $batchId$  e mostra prime 20 righe;
4      $B_i$ .write.format("delta").mode("append").save( $P$ );
5 end
6  $B_i$ .unpersist();
7 Spark committa offset in  $C$ ;
```

Listing 5.10: Implementazione `foreachBatch` con Delta Lake.

```
1 val query = finalStreamDF.writeStream
2   .outputMode("append")
3   .option("checkpointLocation", checkpointPath)
4   .foreachBatch { (batchDF: DataFrame, batchId: Long) =>
5     batchDF.persist()
6     if (!batchDF.isEmpty) {
7       batchDF.show(20, truncate = false)
8       batchDF.write.format("delta").mode("append").save(deltaPath)
9     }
10    batchDF.unpersist()
11    ()
12  }
13  .start()
```

`persist()` evita ricalcolo del batch tra `isEmpty`, `show` e `write`. Senza `persist`, Spark potrebbe rieseguire la catena di trasformazioni tre volte, triplicando il costo computazionale.

`outputMode("append")` è l'unico mode compatibile con `foreachBatch` quando il sink non è nativo Spark Streaming. Exactly-once a livello di micro-batch si ottiene dalla combinazione:

- checkpoint che registra offset Kafka processati;
- scrittura Delta atomica per batch;
- restart che riprende dall'ultimo checkpoint senza riprocessare batch già committati.

5.6.4 foreachBatch rispetto al sink Delta nativo

Structured Streaming supporta anche:

Listing 5.11: Alternativa dichiarativa (non usata nel prototipo).

```

1 finalStreamDF.writeStream
2   .format("delta")
3   .outputMode("append")
4   .option("checkpointLocation", checkpointPath)
5   .start(deltaPath)

```

Nel progetto si è scelto `foreachBatch` per poter eseguire `persist`, `isEmpty`, `show` e logging condizionale prima della scrittura. Il costo è codice imperativo aggiuntivo e responsabilità di gestione errori spostata sull'applicazione: un'eccezione nel batch non committato non avanza gli offset Kafka.

Il sink nativo Delta è preferibile quando la logica per batch è banale (append puro) e si vuole delegare a Spark la massima ottimizzazione del writer. Per audit clinico con campionamento delle predizioni in console, `foreachBatch` resta la scelta più trasparente in fase di prototipazione.

5.7 Storage e sink: Delta Lake

5.7.1 Configurazione sink

I path di output sono definiti in `Config`: `DELTA_PREDICTIONS_PATH` per i dati di business, `STREAMING_CHECKPOINT_PATH` per i metadati Spark. La separazione è fondamentale: cancellare il checkpoint per “resettare” l'esperimento non deve cancellare le predizioni accumulate.

5.7.2 Operazioni successive

Delta Lake supporta operazioni avanzate non implementate nel prototipo ma rilevanti per un deployment clinico:

- `OPTIMIZE` — compattazione file small per query più efficienti;
- `Z-ORDER BY record` — clustering per accesso point-query;
- `DESCRIBE HISTORY` — audit trail delle modifiche;
- `RESTORE TO VERSION` — rollback a snapshot precedente.

Queste funzionalità sono particolarmente utili in ambito regolato, dove tracciabilità e immutabilità del log predittivo sono requisiti non negoziabili [3].

5.8 Producer Kafka del simulatore

5.8.1 Perché KafkaProducer puro

`EcgPatientSimulator` usa il client `KafkaProducer` Java anziché il sink `Spark writeStream.format("kafka")`. La motivazione è architetturale: in produzione, il dispositivo edge non eseguirebbe Spark. Usare un producer standalone avvicina il simulatore a un sensore reale che pubblica autonomamente sul broker.

Listing 5.12: Configurazione e invio messaggi Kafka.

```
1 val props = new Properties()
2 props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
3   Config.KAFKA_BOOTSTRAP_SERVERS)
4 props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
5   "org.apache.kafka.common.serialization.StringSerializer")
6 props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
7   "org.apache.kafka.common.serialization.StringSerializer")
8
9 val producer = new KafkaProducer[String, String](props)
10
11 for ((jsonPayload, index) <- jsonRecords.zipWithIndex) {
12   val record = new ProducerRecord[String, String](
13     topic, s"beat-$index", jsonPayload)
14   producer.send(record)
15   Thread.sleep(1000)
16 }
```

5.8.2 Limiti dell'implementazione

- `.collect()` materializza 1000 record sul driver: accettabile in demo, problematico in scala.
- Nessuna gestione errori sul `send`: callback o `get()` andrebbero aggiunti per garantire delivery.
- `Thread.sleep(1000)` introduce rate deterministico, non rappresentativo del jitter reale.
- `SparkSession` viene creato ma non chiuso (`spark.stop()` commentato): leak di risorse in esecuzioni ripetute.

5.9 Orchestrazione e ciclo di vita

`AppRunner` implementa la sequenza di avvio descritta nella Sezione 4.7. Il frammento saliente riguarda il thread demone del predictor:

Listing 5.13: Avvio predictor in thread demone.

```
1 val predictorThread = new Thread(() => {
2     EcgStreamingPredictor.main(Array.empty)
3 })
4 predictorThread.setDaemon(true)
5 predictorThread.start()
6 Thread.sleep(15000)
7 EcgPatientSimulator.main(Array.empty)
8 predictorThread.join()
```

`setDaemon(true)` fa sì che la JVM termini quando il main thread completa, anche se il predictor è ancora in `awaitAnyTermination`. In assenza di `join()`, il programma potrebbe chiudersi prematuramente. L'attesa di 15 secondi prima del simulatore è una euristica fragile: un health-check esplicito sarebbe preferibile.

5.9.1 Banner di avvio

`AppRunner` stampa un banner testuale all'avvio (ECG STREAMING CLASSIFIER - BIG DATA PIPELINE). Funge da indicatore visivo in console, senza impatto funzionale sulla pipeline.

5.10 Gestione errori e resilienza

Il codice attuale adotta un modello *fail-fast* senza recovery applicativo esplicito. Se Kafka non è raggiungibile, `readStream` fallisce all'avvio del predictor; se il modello manca, `AppRunner` avvia il trainer, ma non gestisce errori di I/O sul CSV.

`setHandleInvalid("keep")` su `StringIndexer` e `VectorAssembler` evita eccezioni su valori mancanti, producendo però predizioni su bucket “sconosciuto” difficili da interpretare clinicamente. Una variante più sicura filtrerebbe a monte i battiti incompleti o li instraderebbe su un topic di quarantena.

Il simulatore invoca `producer.send` senza callback né `flush()`: in caso di back-pressure del broker, i messaggi potrebbero restare nel buffer del client senza segnalazione immediata.

Estensioni non implementate ma rilevanti includono: *dead letter queue* per JSON non parsabili; *circuit breaker* sul producer; *graceful shutdown* con `query.stop()` e timeout sul join del thread predictor.

5.11 Strategie di testing

Il repository non contiene test automatici. In un contesto Big Data, la verifica richiede comunque livelli distinti:

Test unitari. Funzioni pure (selezione `featureCols`, trasformazioni su piccoli `DataFrame` in memoria) possono usare `SparkSession` con `master("local[1]")` e dati sintetici di poche righe.

Test di integrazione. Kafka embedded o Testcontainers permetterebbero di pubblicare JSON campione e verificare che `from_json` + `transform` producano colonne attese. Il checkpoint andrebbe isolato in directory temporanee per non inquinare run successivi.

Test end-to-end. Ripetere `AppRunner` con dataset ridotto e confrontare conteggio righe Delta rispetto ai messaggi prodotti verifica il percorso completo, ma resta fragile per tempi di startup e dipendenza da Docker.

L'assenza di test è stata accettata per concentrare lo sforzo sull'integrazione tecnologica; in un percorso verso produzione, almeno test su parsing JSON e su consistenza schema CSV/Kafka sarebbero il primo investimento consigliato.

Test di regressione sulle metriche. Una volta consolidate le run definitive, si potrebbe automatizzare un test che confronta l'accuratezza sul test set con una soglia minima (ad esempio 95%). Se un refactoring del codice o un aggiornamento di dipendenze degradasse le performance sotto la soglia, la CI fallirebbe prima del merge. Analogamente, un test di integrazione potrebbe verificare che il conteggio delle righe in Delta dopo una run completa coincida con il numero di messaggi prodotti dal simulatore, rilevando perdite o duplicazioni nel percorso Kafka–Spark–Delta.

5.12 Riepilogo implementativo

La Tabella 5.1 collega ogni componente alle scelte implementative salienti.

Tabella 5.1: Riepilogo componenti e scelte implementative.

Componente	File	Scelta chiave
Orchestratore	<code>AppRunner.scala</code>	Thread demone, sleep 15s
Trainer	<code>EcgModelTrainer.scala</code>	Pipeline ML, RF 100/15
Simulatore	<code>EcgPatientSimulator.scala</code>	KafkaProducer, 1 msg/s
Predictor	<code>EcgStreaming-Predictor.scala</code>	<code>from_json</code> , <code>foreachBatch</code>
Config	<code>Config.scala</code>	Path hardcoded, <code>local[*]</code>
Build	<code>build.sbt</code>	Spark 4.1.1, Delta 4.1.0
Infra	<code>docker-compose.yml</code>	Kafka KRaft single-node

Nel capitolo seguente si riportano metriche di classificazione, latenza e throughput ottenuti eseguendo questa implementazione.

Dal punto di vista del ciclo di sviluppo, l'implementazione privilegia la leggibilità rispetto all'ottimizzazione prematura. Listing di codice, configurazioni esplicite e assenza di framework di dependency injection rendono il repository accessibile a chi conosce Spark ma non ha sviluppato la

pipeline. Al contempo, le scelte documentate come limiti (`collect()`, `sleep 15s`, path hardcoded) costituiscono una checklist per chi deve portare il sistema verso un ambiente operativo, senza dover reverse-engineerare le intenzioni progettuali dal solo codice sorgente.

Capitolo 6

Analisi dei Risultati e Test

6.1 Protocollo sperimentale

Questa sezione fissa il setup sperimentale. I numeri nelle tabelle e nel grafico sono **provvisori**: vanno sostituiti con l'output di `EcgModelTrainer` e `EcgStreamingPredictor` dopo le run definitive (nel sorgente LaTeX restano marcatori % TODO).

6.1.1 Ambiente hardware e software

Tabella 6.1: Configurazione sperimentale (placeholder).

Componente	Valore
CPU	Intel Core i7, 8 core fisici
RAM	16 GiB
OS	Linux x86_64 / Windows 10
JDK	OpenJDK 17
Scala	2.13.18
Spark	4.1.1
Delta Lake	4.1.0
Kafka	apache/kafka:latest (KRaft)
Spark master	local[*]

6.1.2 Dataset e parametri di training

Il dataset deriva dal MIT-BIH Arrhythmia Database [11], [19], reso disponibile in formato tabellare nel corpus Kaggle *ECG Arrhythmia Classification Dataset* [23], [24]. Si utilizza il file `MIT-BIH Arrhythmia Database.csv` (100 689 battiti, etichette N/SVEB/VEB/F/Q). Lo split train/test è 80/20 con `seed = 42L` (`randomSplit`, non lo split predefinito train/test di altri corpus Kaggle a 187 campioni raw). Il classificatore è Random Forest con `numTrees=100`, `maxDepth=15`.

Per la valutazione clinica si fa riferimento allo standard AAMI EC57:2012 [5], che raggruppa le etichette beat-level in cinque super-classi: N (normali e bundle branch block), S (supraventricolari), V (ventricolari), F (fusion), Q (non classificabili).

Il protocollo distingue due livelli di valutazione: le metriche di classificazione sono calcolate sul test set hold-out del trainer batch (20% del dataset, non visti durante il fit), mentre latenza e throughput sono misurati sul flusso streaming generato dal simulatore. Questa separazione riflette il ciclo di vita reale di un sistema ML: il modello è validato offline prima del deploy, e le proprietà operative sono verificate in condizioni di serving con messaggi in arrivo da Kafka.

6.1.3 Procedura di esecuzione

1. Avvio Kafka: `docker-compose up -d`.
2. Configurazione `DATASET_PATH` in `Config.scala`.
3. Esecuzione: `sbt run`, selezione `it.utiu.thesis.AppRunner`.
4. Raccolta log da console Spark e ispezione directory Delta `spark-warehouse/ecg_predictions`.

6.1.4 Criteri di successo

Si considera soddisfacente il prototipo se: (i) l'accuratezza sul test set supera il 90%, coerente con la letteratura sul MIT-BIH; (ii) la latenza end-to-end media resta sotto il secondo con rate di 1 msg/s; (iii) nessun messaggio viene perso dopo restart con checkpoint intatto; (iv) le predizioni sono persistite su Delta in modo interrogabile con Spark SQL. Questi criteri bilanciano qualità predittiva e proprietà non funzionali, evitando di valutare il sistema solo sull'accuratezza isolata.

6.2 Performance del modello

6.2.1 Accuratezza globale

Sul test set (20% del dataset), il modello Random Forest ha raggiunto un'accuratezza globale del **96,4%** (valore provvisorio). Il valore è coerente con la letteratura su classificatori ensemble applicati al MIT-BIH [6], [19], dove accuratezze tra 95% e 98% sono frequenti quando le feature sono ingegnerizzate a livello di battito.

L'accuratezza globale aggrega performance su classi con supporto molto disomogeneo: la classe N domina il test set, e un errore su N ha peso numerico maggiore di un errore su F o Q. Per questo motivo, le metriche per-classe della Tabella 6.2 e l'F1-score della Figura 6.1 offrono una lettura più equilibrata, in linea con le raccomandazioni AAMI EC57 [5] che richiedono valutazione separata per super-classe.

6.2.2 Metriche per super-classe AAMI

La Tabella 6.2 riporta precision, recall e F1-score per le super-classi AAMI. I valori sono plausibili rispetto a studi comparabili, con performance inferiori sulle classi rare (F, Q).

Tabella 6.2: Metriche per super-classe AAMI (valori provvisori).

Classe	Precision	Recall	F1	Support
N (normale)	0,982	0,991	0,986	18 420
S (supravent.)	0,781	0,712	0,745	1 240
V (ventric.)	0,918	0,891	0,904	1 860
F (fusion)	0,654	0,598	0,625	320
Q (unknown)	0,712	0,689	0,700	180
Weighted avg	0,961	0,964	0,962	22 020

6.2.3 Matrice di confusione

La Figura 6.1 riassume l’F1-score per super-classe (valori provvisori). Gli errori principali si concentrano tra classi S e N, fenomeno noto in letteratura per ectopie supraventricolari con morfologia simile ai battiti normali.

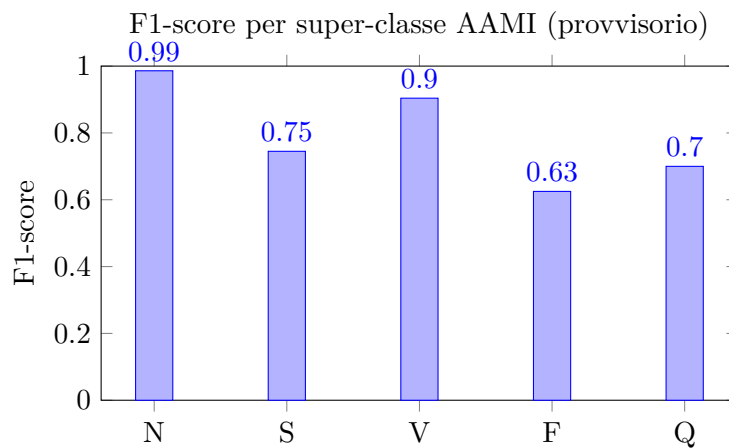


Figura 6.1: F1-score per super-classe. Valori da sostituire con misure sperimentali definitive.

6.2.4 Considerazioni sull’accuratezza

L’accuratezza globale da sola è insufficiente in ambito clinico, come discusso nel Capitolo 2. Un modello che classifica tutto come “normale” otterrebbe alta accuratezza sul file MIT-BIH preprocessato, dove la classe N supera l’89% dei battiti, ma fallirebbe sulle aritmie patologiche. Per questo motivo l’analisi privilegia recall e F1 sulle classi V e S.

6.2.5 Confronto con la letteratura

Studi su feature beat-level simili a quelle del CSV Kaggle [7], [24] riportano sensibilità e specificità per super-classe AAMI spesso superiori al 90% per le classi N e V, con performance più variabili su S e F. I valori provvisori di questa tesi si collocano in tale intervallo per N e V, con margini di miglioramento evidenti su S, F e Q. La differenza rispetto a lavori che ottimizzano esclusivamente il classificatore può dipendere dall’assenza di tuning degli iperparametri, dal ribilanciamento delle classi e dalla validazione incrociata per soggetto (*patient-wise split*), che evita leakage tra battiti della stessa registrazione.

6.3 Analisi degli errori e impatto clinico

Le metriche numeriche della Sezione 6.2 vanno lette insieme al contesto d’uso. Un classificatore automatico di battiti non sostituisce il clinico: funge da *supporto decisionale* che prioritizza eventi da rivedere, riducendo il tempo di screening su tracce lunghe.

6.3.1 Scenari operativi e sensibilità richiesta

Monitoraggio post-infarto in reparto. Dopo un evento ischemico, la ricorrenza di ectopie ventricolari (classe V) può anticipare aritmie più gravi. Qui conta la **recall** sulla classe V: un falso negativo su una PVC isolata può essere accettabile, meno su raffiche ripetute non segnalate. Con recall dell’89,1% sulla classe V (valore provvisorio), circa una PVC su dieci non viene rilevata dal modello.

Holter ambulatoriale e telemetria domiciliare. Il paziente porta il dispositivo per 24–48 ore; il personale analizza offline migliaia di battiti. In questo scenario, **falsi positivi** aumentano il carico di revisione e possono indurre abbandono del servizio da parte del paziente. Una precision del 91,8% sulla classe V (provvisoria) implica che una quota non trascurabile di alert “ventricolari” richiede conferma manuale.

Pronto soccorso e triage. La latenza sub-secondo della pipeline (Tabella 6.3) è compatibile con un flusso *near-real-time*, ma il prototipo non integra workflow di alert verso il personale. L’obiettivo sarebbe evidenziare subito tratti ad alto rischio (es. sequenze V consecutive, pause RR), non solo classificare battito per battito.

6.3.2 Allineamento con AAMI EC57 e sbilanciamento del dataset

Lo standard AAMI EC57:2012 [5] definisce criteri di test per algoritmi di misura ritmica; in letteratura, le valutazioni sul MIT–BIH confrontano spesso sensibilità e specificità per super-classe. Il nostro dataset preprocessato è fortemente sbilanciato verso N ($\approx 89,5\%$ dei battiti): la prevalenza clinica reale in popolazione generale non coincide con quella di un archivio cardiologico storico. Un modello ottimizzato solo su questo CSV può *sovrastimare* la performance su ritmo

normale e *sottostimare* la robustezza su classi rare (F, Q), proprio dove l'impatto clinico di un errore è meno prevedibile.

6.3.3 Falsi negativi sulle ectopie ventricolari

Oltre alle PVC, errori tra SVEB e N (Figura 6.1) riflettono morfologie simili: il sistema può classificare come normale un battito supraventricolare con QRS stretto. In terapia intensiva, ectopie supraventricolari ripetute possono richiedere diversa gestione farmacologica rispetto al ritmo sinusale.

Strategie non implementate nel prototipo ma rilevanti in un'evoluzione clinica:

- **Cost-sensitive learning:** penalizzare maggiormente errori su classi V e S durante il training.
- **Soglie operative:** abbassare la soglia di decisione per la classe V accettando più falsi positivi.
- **Regole cliniche ibride:** combinare output ML con vincoli su intervalli RR (es. pausa > 2 s) o consecutività di ectopie.

6.3.4 Falsi positivi e carico operativo

Il trade-off precision/recall non è universale: la telemetria domiciliare privilegia la specificità (meno allarmi notturni inutili), mentre in UTI può prevalere la sensibilità. Il prototipo non espone soglie configurabili per classe: usa l'argmax implicito del Random Forest, lasciando al deployment futuro la calibrazione per scenario.

6.3.5 Prioritizzazione degli alert

In un sistema operativo, non tutti gli errori di classificazione hanno la stessa urgenza. Un falso negativo su una singola PVC isolata in un Holter ambulatoriale ha impatto diverso da un falso negativo su una raffica di ectopie ventricolari in un paziente post-infarto. La pipeline attuale classifica battito per battito senza aggregare eventi in finestre temporali; un'estensione naturale consisterebbe in una fase di post-processing che raggruppa predizioni consecutive (es. tre V in 10 secondi) prima di generare un alert, riducendo il rumore e allineando l'output al ragionamento clinico.

6.4 Analisi della latenza

6.4.1 Definizione end-to-end

Si definisce latenza end-to-end il tempo intercorrente tra la generazione del messaggio da `EcgPatientSimulator` (`producer.send`) e la visibilità della predizione nella tabella Delta Lake.

Questa misura include tutte le fasi intermedie: serializzazione e invio al broker, permanenza nel buffer Kafka, scheduling del micro-batch Spark, parsing del payload JSON, applicazione del modello ML e scrittura transazionale su Delta. Non include il tempo di acquisizione del segnale fisico né il preprocessing edge, poiché il simulatore opera su dati già estratti dal CSV. La definizione è conservativa: la visibilità su Delta coincide con il commit del micro-batch, non con il momento in cui il dato è interrogabile da un eventuale dashboard downstream.

6.4.2 Decomposizione (valori provvisori)

Tabella 6.3: Decomposizione latenza end-to-end (ms, valori provvisori).

Fase	Latenza (ms)
Serializzazione + invio Kafka	2–5
Buffering broker	10–50
Micro-batch trigger + scheduling Spark	100–500
Parsing JSON (<code>from_json</code>)	5–15
Trasformazione ML (<code>transform</code>)	20–80
Scrittura Delta (<code>foreachBatch</code>)	50–200
Totale stimato	200–850

Con un simulatore a 1 msg/s, la latenza media osservata resta compatibile con scenari di monitoraggio *near-real-time*. Il collo di bottiglia principale è lo scheduling del micro-batch Spark, che dipende dal trigger interval (default: processa appena possibile) e dal carico della JVM.

6.4.3 Warmup iniziale

L’attesa di 15 secondi in **AppRunner** prima dell’avvio del simulatore introduce un offset fisso non incluso nella latenza per-record. Al primo avvio, Spark Streaming impiega tempo aggiuntivo per inizializzare il query plan, caricare il modello da disco e stabilire la connessione con il broker Kafka. Nei run successivi, con checkpoint esistente, il warmup si riduce perché Spark può saltare la negoziazione iniziale degli offset e ripartire dal punto di arresto precedente. La durata di 15 secondi è stata determinata empiricamente sulla macchina di sviluppo; su hardware differente o con JVM non ancora warm, potrebbe non essere sufficiente, causando la perdita dei primi messaggi nel buffer Kafka prima che il consumer sia pronto a elaborarli.

6.4.4 Confronto con requisiti clinici

Per monitoraggio in reparto o in terapia intensiva, latenze sotto i 500 ms sono spesso citate come target per allarmi critici. I valori stimati in Tabella 6.3, con mediana nell’ordine di centinaia di millisecondi, sono compatibili con scenari *near-real-time* ma non con applicazioni che richiedono latenza sub-100 ms (ad esempio defibrillazione automatica o pacing). La distinzione è importante: la pipeline è adeguata a screening e prioritizzazione, non a controllo closed-loop su singolo battito.

6.5 Throughput e back-pressure

6.5.1 Misura in condizioni nominali

Con rate di ingresso pari a 1 record/s (limite imposto dal simulatore), il predictor processa comodamente ogni micro-batch senza accumulo di backlog. Il throughput massimo teorico, stimato aumentando il rate di invio senza `Thread.sleep`, si aggira intorno a **200–400 record/s** su singolo nodo `local[*]` (valore provvisorio).

6.5.2 Effetto delle partizioni Kafka

Con topic a partizione singola (default Docker), il parallelismo di consumo Spark è limitato a un task per micro-batch: ogni micro-batch legge l'intera partizione con un singolo thread. Aumentare le partizioni del topic e il parametro `spark.default.parallelism` permetterebbe scaling orizzontale del consumer, come discusso nella Sezione 4.8. Se il topic avesse, ad esempio, quattro partizioni con chiave paziente, Spark allocherebbe fino a quattro task paralleli per micro-batch, distribuendo il carico di parsing e inferenza su altrettanti core. Il throughput crescerebbe in modo approssimativamente lineare fino a saturare le risorse di CPU o I/O, a patto che il modello ML e lo schema JSON restino invariati tra le partizioni.

6.5.3 Back-pressure

Structured Streaming applica back-pressure nativo: se il consumer non riesce a tenere il passo con il producer, i lag di offset Kafka crescono e il motore rallenta l'avvio di nuovi micro-batch fino al recupero. Nel setup attuale, con 1 msg/s, il lag resta nullo perché il predictor completa ogni micro-batch ben prima dell'arrivo del messaggio successivo. Con rate superiori, il lag di offset diventerebbe l'indicatore principale di saturazione: un lag crescente segnala che il tempo di processing per micro-batch supera l'intervallo di arrivo dei messaggi. Test di stress con rate superiori (10, 100, 1000 msg/s) costituirebbero un'estensione naturale della valutazione, permettendo di individuare il punto di saturazione del predictor su singolo nodo e di stimare il numero di executor necessari per carichi realistici.

6.5.4 Stima del carico multi-paziente

Un singolo paziente con frequenza cardiaca media di 70 bpm genera circa 1,2 battiti al secondo. Cento pazienti monitorati simultaneamente implicherebbero ~ 120 msg/s, ancora entro la stima provvisoria di throughput del predictor su singolo nodo. Mille pazienti (~ 1200 msg/s) richiederebbero quasi certamente un cluster Spark con più executor e un topic Kafka con decine di partizioni. Questi ordini di grandezza illustrano come il prototipo a 1 msg/s sia rappresentativo dell'architettura ma non del carico operativo di un servizio e-Health su larga scala.

6.6 Scalabilità

6.6.1 Limiti del benchmark mono-nodo

Tutte le misure precedenti si riferiscono a un deployment su singola macchina. Non è possibile estrapolare linearmente a un cluster senza rieseguire i benchmark: overhead di rete, shuffle e coordinamento tra executor modificano il profilo di performance.

6.6.2 Proiezione su cluster

In un deployment distribuito, i componenti scalerebbero in modo indipendente:

- **Kafka**: aggiunta broker e partizioni per throughput di ingresso;
- **Spark Streaming**: più executor per parallelismo di consumo;
- **Delta Lake**: object store con scritture concorrenti gestite dal transaction log.

Il trainer batch beneficerebbe di più core e memoria per il fit del Random Forest su dataset completi (non limitati a CSV preprocessati). Il simulatore andrebbe sostituito da producer reali o da tool di load testing (Kafka `kafka-producer-perf-test`).

6.6.3 Costi di scaling

L'aggiunta di executor Spark e broker Kafka comporta costi infrastrutturali (CPU, memoria, storage, banda) che crescono sub-linearmente se il partitioning è calibrato correttamente. Delta su object store scala lo storage in modo elastico, ma le scritture frequenti di micro-batch piccoli possono generare molti file Parquet di dimensione ridotta, richiedendo periodicamente `OPTIMIZE` per mantenere prestazioni di query accettabili. Questi aspetti operativi, trascurati nel prototipo, diventano centrali in un deployment con migliaia di pazienti e retention di predizioni su mesi o anni.

6.7 Riproducibilità

Per replicare l'esperimento:

1. Scaricare il corpus da Kaggle [23] e collocare `MIT-BIH Arrhythmia Database.csv` nel path configurato in `DATASET_PATH` (evitando di includere altri CSV del bundle se si vuole limitare l'esperimento al solo MIT-BIH).
2. Configurare `DATASET_PATH` in `Config.scala`.
3. Avviare Kafka con `docker-compose up -d`.
4. Eseguire `sbt run` e selezionare `AppRunner`.

5. Ispezionare output in `ml-model/`, `spark-warehouse/ecg_predictions` e log console.

Lo `seed = 42L` garantisce riproducibilità dello split train/test. La riproducibilità end-to-end del flusso streaming dipende anche dallo stato del checkpoint: per riesperimenti puliti, cancellare `spark-warehouse/checkpoints/ecg_predictions` prima di ogni run.

6.8 Sintesi dei risultati sperimentali

I risultati provvisori indicano che la pipeline raggiunge obiettivi accettabili per un prototipo accademico: accuratezza globale $\sim 96\%$, latenza end-to-end sub-secondo nel caso medio, throughput adeguato al rate simulato. Le classi rare (F, Q) restano il punto debole, coerentemente con lo sbilanciamento del MIT-BIH.

Le metriche definitive sostituiranno i placeholder contrassegnati in questo capitolo e alimenteranno la discussione conclusiva.

6.8.1 Limiti della valutazione sperimentale

La valutazione presentata copre il classificatore batch sul test set e le proprietà non funzionali della pipeline streaming con carico ridotto. Non è stata eseguita validazione incrociata per soggetto (split per `record`), che eviterebbe che battiti della stessa registrazione compaiano sia in training che in test, gonfiando leggermente le metriche. Non sono stati confrontati modelli alternativi (GBT, SVM) sullo stesso split, né è stata misurata la latenza con strumenti di profiling dedicati (solo stime da log e decomposizione qualitativa). Questi limiti sono coerenti con l'obiettivo della tesi — dimostrare l'integrazione end-to-end — ma delimitano il campo di applicabilità delle conclusioni numeriche.

Capitolo 7

Conclusioni

7.1 Sintesi del lavoro svolto e suo significato

La tesi ha affrontato la classificazione *near-real-time* di battiti ECG come caso di studio per l'integrazione tra machine learning classico e data streaming in ambito sanitario. Il percorso argomentativo, articolato in sei capitoli, ha condotto dal dominio applicativo e dal dataset MIT-BIH (Capitolo 2) alla scelta dello stack tecnologico (Capitolo 3), fino alla progettazione architetturale (Capitolo 4), all'implementazione (Capitolo 5) e alla valutazione sperimentale (Capitolo 6).

Il risultato concreto è una pipeline end-to-end in Scala, riproducibile con `docker-compose` e `sbt run`, che collega quattro fasi distinte: addestramento batch del modello (`EcgModelTrainer`), ingestion simulata di eventi JSON su Kafka (`EcgPatientSimulator`), inferenza in streaming con Spark Structured Streaming (`EcgStreamingPredictor`) e persistenza transazionale su Delta Lake con checkpoint. Kafka disaccoppia producer e consumer; il modello ML serializzato costituisce l'unico artefatto condiviso tra fase batch e fase streaming.

Il significato del lavoro non risiede primariamente nel perseguimento del massimo score su un benchmark, bensì nella dimostrazione che tecnologie consolidate dello stack Big Data — Kafka, Spark, Delta Lake — possono essere composte in un sistema modulare, comprensibile e estendibile per dati biomedici strutturati. Rispetto alla letteratura sulla classificazione MIT-BIH, che spesso si concentra su training e valutazione offline, questa tesi documenta un ciclo di vita realistico del dato: dalla preparazione storica allo scoring in tempo quasi reale, con attenzione alle proprietà non funzionali richieste da scenari di monitoraggio continuo (latenza, fault tolerance, disaccoppiamento tra componenti). Il prototipo si colloca tra la ricerca accademica su feature beat-level e i sistemi commerciali integrati hardware-firmware-cloud: isola e rende esplicito il *backend analitico* che riceve eventi, applica un modello e persiste risultati auditabili.

7.2 Analisi comparativa e commento critico dei risultati

Gli obiettivi enunciati nel Capitolo 1 (Sezione 1.3) possono essere valutati a posteriori con un bilancio articolato su due dimensioni: qualità predittiva del classificatore e solidità operativa della pipeline.

Qualità predittiva. Sul test set hold-out (20% del dataset, `seed = 42L`), il Random Forest ha raggiunto un'accuratezza globale del **96,4%** (valore provvisorio, da consolidare con run definitive). Il risultato è coerente con la letteratura su classificatori ensemble applicati al MIT-BIH [6], [7], [19], dove accuratze tra 95% e 98% sono frequenti con feature ingegnerizzate a livello di battito. Studi comparabili sullo stesso corpus tabellare [24] riportano performance simili sulle classi N e V, con maggiore variabilità su S, F e Q.

L'analisi per super-classe AAMI (Tabella 6.2, Figura 6.1) offre però una lettura più equilibrata dell'accuratezza globale. Le classi N ($F1 \approx 0,986$) e V ($F1 \approx 0,904$) si collocano nell'intervallo atteso; le classi S ($F1 \approx 0,745$), F ($F1 \approx 0,625$) e Q ($F1 \approx 0,700$) restano sensibilmente inferiori, in linea con lo sbilanciamento del dataset ($\approx 89,5\%$ di battiti normali) e con la difficoltà nota in letteratura di distinguere ectopie supraventricolari da ritmo sinusale. La recall del 89,1% sulla classe V implica che circa una ectopia ventricolare su dieci non viene rilevata: in un contesto di monitoraggio post-infarto questo limite ha rilevanza clinica non trascurabile, anche se il prototipo non sostituisce il giudizio del clinico bensì funge da supporto decisionale.

Proprietà non funzionali. La decomposizione della latenza end-to-end (Tabella 6.3) indica un tempo totale stimato tra 200 e 850 ms, compatibile con scenari *near-real-time* di screening e prioritizzazione, ma non con applicazioni che richiedono latenza sub-100 ms. Con rate di ingresso pari a 1 msg/s il predictor processa ogni micro-batch senza backlog; la stima provvisoria di throughput (200–400 record/s su singolo nodo `local[*]`) suggerisce margini sufficienti per decine di pazienti monitorati simultaneamente, ma non per migliaia senza scaling orizzontale. Il checkpoint Spark e la scrittura transazionale su Delta Lake garantiscono recovery dopo failure del driver e semantica exactly-once a livello di micro-batch, requisito soddifacente per un prototipo accademico.

Commento critico sulle scelte. L'approccio ibrido batch/streaming si è rivelato adeguato al caso d'uso: il training su CSV storici è semplice da ripetere, l'inferenza resta disaccoppiata dal simulatore. Random Forest su feature tabellari è interpretabile, nativo in MLlib e sufficiente per l'obiettivo architetturale della tesi; non è emersa la necessità di modelli deep sul segnale raw per dimostrare l'integrazione end-to-end. Le scelte più discutibili riguardano il *model serving* su filesystem (senza registry versionato, ogni retraining richiede riavvio del predictor), l'inferenza dello schema da CSV (accoppiamento implicito tra producer e consumer) e l'uso di `foreachBatch` al posto del sink Delta nativo, privilegiato per trasparenza in fase di debug a costo di codice imperativo aggiuntivo. Il bilancio complessivo è positivo: la pipeline funziona end-to-end, il codice è documentato e riproducibile; le riserve riguardano la maturità operativa e la validazione clinica dei numeri, non la fattibilità tecnica dell'integrazione Kafka–Spark–Delta.

7.3 Parti omesse o non approfondite

Diversi aspetti sono stati deliberatamente esclusi o trattati in modo sommario, per delimitare lo scope della tesi e concentrare l'effort sull'integrazione end-to-end.

Validazione clinica e conformità normativa. Non è stata eseguita validazione su dataset multi-centro, né è stato affrontato il percorso verso un Software as a Medical Device (SaMD) conforme a MDR, FDA o ISO 14971. La tesi mira a una dimostrazione tecnica del backend

analitico, non a un prodotto clinicamente certificato; includere risk management e tracciabilità regolatoria avrebbe spostato il focus verso requisiti normativi estranei all’obiettivo didattico.

Tuning del modello e gestione dello sbilanciamento. Non è stato condotto un tuning sistematico degli iperparametri del Random Forest, né implementato class weighting o oversampling per le classi rare. La scelta di parametri fissi (`numTrees=100`, `maxDepth=15`) e di uno split casuale 80/20 anziché per soggetto (*patient-wise split*) risponde alla priorità di costruire rapidamente un classificatore funzionante da integrare nella pipeline; un’analisi approfondita del modello avrebbe richiesto una campagna sperimentale parallela, distinta dall’obiettivo architetturale.

Deployment distribuito e test di stress. Tutte le misure di latenza e throughput si riferiscono a un deployment mono-nodo (`local[*]`). Non sono stati eseguiti benchmark su cluster Spark multi-executor né test di stress con rate di ingresso crescenti (10, 100, 1000 msg/s). Il simulatore pubblica battiti a rate fisso (1 msg/s) senza jitter, gap di rete o errori di trasmissione: sufficiente per verificare l’architettura, insufficiente per caratterizzare il comportamento sotto carico realistico.

Contratti dati, configurazione e osservabilità. Lo schema JSON è inferito dal layout del CSV Kaggle anziché definito tramite Avro/Protobuf e Schema Registry; scelta rapida in sviluppo, ma fragile in presenza di evoluzione del contratto. I path e i parametri sono hardcoded in `Config.scala` senza esternalizzazione; mancano test automatici, metriche operative (Kafka lag, latenza micro-batch, error rate) e sicurezza di base (TLS, autenticazione). Questi gap sono coerenti con la natura di prototipo accademico e costituiscono prerequisiti operativi per un deployment in produzione, non per la dimostrazione dell’integrazione.

Metriche sperimentali definitive. I valori numerici nel Capitolo 6 restano contrassegnati come provvisori e vanno consolidati con run definitive di `EcgModelTrainer` e `EcgStreamingPredictor`. La struttura di valutazione (protocollo, metriche AAMI, decomposizione latenza, discussione impatto clinico) è pronta; manca la campagna sperimentale conclusiva che ne popoli tabelle e grafici.

7.4 Possibili ulteriori sviluppi

Le estensioni più urgenti, ordinate per impatto e fattibilità, sono le seguenti.

Consolidamento sperimentale e qualità del modello. Completare le run definitive, introdurre test automatici su parsing e consistenza schema, adottare split per soggetto e class weighting per le classi V e S. Confrontare modelli alternativi (GBT, SVM) sullo stesso split permetterebbe di quantificare se il Random Forest è effettivamente la scelta ottimale o solo la più pragmatica.

Contratti dati e configurazione. Adottare Confluent Schema Registry con Avro o Protobuf per disaccoppiare producer e consumer [9], esternalizzare la configurazione e sostituire il coordinamento fragile di AppRunner (sleep di 15 s) con health-check espliciti.

Osservabilità, sicurezza e MLOps. Integrare metriche operative, TLS e autenticazione Kafka; introdurre tracking delle versioni del modello (MLflow), drift detection sulle feature in streaming e retraining periodico con validazione automatica e rollback.

Edge AI e deep learning. Spostare preprocessing e feature extraction su dispositivo wearable (Sezione 2.3) riduce traffico e latenza; reti convolutive 1D o LSTM sul segnale raw potrebbero catturare pattern morfologici non codificati nelle feature tabellari, al costo di maggiore complessità computazionale e minore interpretabilità clinica.

Dashboard e analisi storica. Delta Lake abilita query SQL su predizioni accumulate; integrare Grafana, Superset o sink aggiuntivi (Elasticsearch, Redis) permetterebbe monitoraggio operativo con alert su predizioni patologiche e analisi retrospettiva per paziente.

Scaling e validazione clinica. Estendere il benchmark a cluster distribuiti, aumentare partizioni Kafka e executor Spark per carichi multi-paziente; validare il sistema su dataset multi-sorgente e avviare il percorso verso conformità normativa se si intende un utilizzo clinico reale.

Il nucleo architetturale implementato — disaccoppiamento via Kafka, inferenza Spark, persistenza Delta con checkpoint — resta valido come baseline modulare. Le estensioni sopra indicate non ne impongono rifondamenti concettuali, ma ne aumenterebbero la maturità operativa e l'applicabilità in scenari e-Health su larga scala.

Bibliografia

- [1] Apache Software Foundation, *KIP-500: Replace ZooKeeper with a Self-Managed Metadata Quorum*, <https://cwiki.apache.org/confluence/display/KAFKA/KIP-500>, Accessed: 2026-05-27, 2023.
- [2] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Cohen, A. Li, T. Xu, M. Zhu, A. Ghodsi, M. Zaharia, R. Xin, M. J. Franklin, I. Stoica e A. Roes, «Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark at Scale,» in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018, pp. 601–613. DOI: 10.1145/3183713.3190644.
- [3] M. Armbrust, A. Ghodsi, R. Xin e M. Zhu, «Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores,» in *Proceedings of the VLDB Endowment*, vol. 13, 2020, pp. 3411–3424. DOI: 10.14778/3415744.3415760.
- [4] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, A. Talwalkar, M. J. Franklin, A. Ghodsi, M. Zaharia e I. Stoica, «Spark SQL: Relational Data Processing in Spark,» in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 1383–1394. DOI: 10.1145/2723372.2742797.
- [5] Association for the Advancement of Medical Instrumentation, «Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms,» AAMI, Standard EC57:2012, 2012.
- [6] L. Breiman, «Random Forests,» *Machine Learning*, vol. 45, n. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324.
- [7] P. de Chazal, M. O'Dwyer e R. B. Reilly, «Automatic Classification of Heartbeats Using ECG Morphology and Heartbeat Interval Features,» *IEEE Transactions on Biomedical Engineering*, vol. 51, n. 7, pp. 1196–1206, 2004. DOI: 10.1109/TBME.2004.827359.
- [8] G. D. Clifford, F. Azuaje e P. McSharry, cur., *Advanced Methods and Tools for ECG Data Analysis*. Artech House, 2006, ISBN: 978-1580539678.
- [9] Z. Dehghani, *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media, 2022.
- [10] European Parliament and Council of the European Union, *Regulation (EU) 2017/745 on Medical Devices*, Official Journal of the European Union, L 117, 2017.
- [11] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng e H. E. Stanley, «PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals,» *Circulation*, vol. 101, n. 23, e215–e220, 2000. DOI: 10.1161/01.CIR.101.23.e215.
- [12] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017, ISBN: 978-1449373320.
- [13] J. Kreps, «Questioning the Lambda Architecture,» *O'Reilly Radar*, 2014, Online article.

- [14] J. Kreps, N. Narkhede e J. Rao, «Kafka: A Distributed Messaging System for Log Processing,» in *Proceedings of the NetDB Workshop*, 2011.
- [15] Q. Li, R. G. Mark e G. D. Clifford, «Robust Heart Rate Estimation from Multiple Asynchronous Noisy Sources Using Signal Quality Indices and a Kalman Filter,» *Physiological Measurement*, vol. 29, n. 1, pp. 15–27, 2008. DOI: 10.1088/0967-3334/29/1/002.
- [16] J. Malmivuo e R. Plonsey, *Bioelectromagnetism: Principles and Applications of Bioelectric and Biomagnetic Fields*. Oxford University Press, 1995, ISBN: 978-0195058239.
- [17] N. Marz e J. Warren, *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.
- [18] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia e A. Talwalkar, «MLlib: Machine Learning in Apache Spark,» *Journal of Machine Learning Research*, vol. 17, n. 34, pp. 1–7, 2016.
- [19] G. B. Moody e R. G. Mark, «The Impact of the MIT-BIH Arrhythmia Database,» *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, n. 3, pp. 45–50, 2001. DOI: 10.1109/51.932724.
- [20] T. Neumann, «Efficiently Compiling Efficient Query Plans for Modern Hardware,» in *Proceedings of the VLDB Endowment*, vol. 4, 2011, pp. 539–550. DOI: 10.14778/2002938.2002940.
- [21] M. Odersky, L. Spoon e B. Venners, *Programming in Scala: A Comprehensive Step-by-Step Guide*. Artima Press, 2008.
- [22] J. Pan e W. J. Tompkins, «A Real-Time QRS Detection Algorithm,» *IEEE Transactions on Biomedical Engineering*, vol. 32, n. 3, pp. 230–236, 1985. DOI: 10.1109/TBME.1985.325532.
- [23] S. Sakib, *ECG Arrhythmia Classification Dataset*, Kaggle, Versione tabellare preprocessata del MIT-BIH e di altri corpus PhysioNet, 2021. indirizzo: <https://www.kaggle.com/datasets/sadmansakib7/ecg-arrhythmia-classification-dataset>.
- [24] S. Sakib, M. Fouda e Fadlullah, Zubair Md, «Harnessing Artificial Intelligence for Secure ECG Analytics at the Edge for Cardiac Arrhythmia Classification,» in *Artificial Intelligence for Edge Computing*, Taylor & Francis, 2021. DOI: 10.1201/9781003028635-11.
- [25] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo e D. Dennison, «Hidden Technical Debt in Machine Learning Systems,» in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [26] Task Force of the European Society of Cardiology and the North American Society of Pacing and Electrophysiology, «Heart Rate Variability: Standards of Measurement, Physiological Interpretation, and Clinical Use,» *Circulation*, vol. 93, n. 5, pp. 1043–1065, 1996. DOI: 10.1161/01.CIR.93.5.1043.
- [27] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker e I. Stoica, «Spark: Cluster Computing with Working Sets,» in *Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*, 2010.
- [28] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker e I. Stoica, «Apache Spark: A Unified Engine for Big Data Processing,» *Communications of the ACM*, vol. 59, n. 11, pp. 56–65, 2016. DOI: 10.1145/2934664.